

**AI-POWERED MULTIMODAL E-COMMERCE SEARCH
SYSTEM USING FAISS AND HYBRID RETRIEVAL**

Project Report

by

MANGIPUDI SANYASI RAO,

Enroll. No. TU6243204211024

Under the Guidance of

Ms.V. KIRUTHIGA MCA., M.Phil., Ph.D.,

Assistant Professor,

School of Computer Science

in partial fulfillment of the requirements for the award of the degree

MASTER OF COMPUTER APPLICATIONS



Submitted to the

SCHOOL OF COMPUTER SCIENCE

FACULTY OF SCIENCES

TAKSHASHILA UNIVERSITY

Ongur, Tindivanam.

APRIL 2026

TAKSHASHILA UNIVERSITY

FACULTY OF SCIENCES

SCHOOL OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the project entitled “**AI-POWERED MULTIMODAL E-COMMERCE SEARCH SYSTEM USING FAISS AND HYBRID RETRIEVAL**” is a Bonafide work of MANGIPUDI SANYASI RAO, Enrollment Number: **TU6243204211024**, who carried out the project under my supervision in the partial fulfillment of the requirement for the degree of **MASTER OF COMPUTER APPLICATIONS** during the year **2025 - 2026**. I further certify that the project work done by him/her has not previously formed the basis for the award to the candidate or to anyone else of any degree or any similar title.

School Incharge

Dr.T. VIJAYAKUMAR, MCA, Ph.D.,

Assistant Professor

Project Guide

Ms.V. KIRUTHIGA, MCA., M.Phil., Ph.D.,

Assistant Professor

Submitted to Viva-Voce examination held on _____

Internal Examiner

External Examiner

Place: Ongur

Date:

DECLARATION

I, MANGIPUDI SANYASI RAO, hereby declare that my project entitled “**AI-POWERED MULTIMODAL E-COMMERCE SEARCH SYSTEM USING FAISS AND HYBRID RETRIEVAL**” is done during **2025-2026** for the award of **Master of Computer Applications** under **School of Computer Science** of **Takshashila University** as a part of my curriculum.

I further declare that this project or a part of this project has not been submitted elsewhere for the award of any other degree.

Place: Ongur

Date:

MANGIPUDI SANYASI RAO

(Reg., No. TU6243204211024)

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without mentioning the people who made it possible and with whose constant guidance and encouragement, I secured the success. I take privilege to express my gratitude and respect to all those who helped me in the completion of this project work.

I express my gratefulness to our most honourable **Shri.M. Dhanasekaran**, Chancellor, **Dr. Prof.Vivek Inder Kochhar**, Vice-Chancellor, **Dr.S. Senthil**, Registrar, and **Dr.R. Subramanian**, Dean – Academic Affairs, for their sincere endeavor in educating us in their premier institution.

I express my sincere thanks to our respected **Dr.S. Deepa**, Dean, Faculty of Sciences, for motivating in completing the project.

I owe a great sense of gratitude to **Dr.T. Vijayakumar**, School Incharge, School of Computer Science, for timely and valuable suggestions during this project work.

I deem it a pleasant duty to place my sincere thanks and heartfelt gratitude to my **Guide Ms.V. kiruthiga**, Assistant Professor, School of Computer Science, who is the source of inspiration and encouragement and provided unstinted guidance throughout this project work. I spontaneously hold in high esteem to all my faculty members, **School of Computer Science**, for bringing out this project successfully with their enthusiastic guidance and making all necessary arrangements.

At the outset I extend my deepest love and gratitude to **My Parents and Family Members** who have built my carrier and backed me in my difficulties.

I like to thank all **My Friends and Well Wishers** who have helped me by their valuable suggestions in completing this project successfully.

MANGIPUDI SANYASI RAO

(Reg No.: **TU6243204211024**)

ABSTRACT

The rapid expansion of e-commerce platforms has resulted in massive product catalogs containing both structured and unstructured data, including textual descriptions and visual content. Traditional search systems in e-commerce primarily rely on keyword-based matching, which often fails to capture the semantic intent of user queries and does not effectively utilize visual information. This leads to inaccurate search results, reduced user satisfaction, and limited recommendation capabilities.

This project presents a Multimodal AI-Powered E-Commerce Product Recommendation System using FAISS and hybrid retrieval with LLM-based explanation generation, designed to overcome the limitations of conventional search systems. The proposed system integrates semantic text understanding, image-based similarity retrieval, hybrid ranking mechanisms, and explainable AI techniques into a unified architecture.

For textual data processing, Sentence Transformers are employed to generate dense semantic embeddings from product titles and descriptions, enabling meaning-based search rather than exact keyword matching. For visual understanding, OpenCLIP (Vision Transformer) is utilized to extract image embeddings, allowing the system to identify visually similar products. Both text and image embeddings are indexed using Facebook AI Similarity Search (FAISS), ensuring efficient and scalable nearest-neighbor retrieval in high-dimensional vector spaces.

A hybrid retrieval mechanism is implemented by combining semantic similarity scores with keyword-based boosting techniques to improve ranking accuracy. Additionally, an LLM-based explanation module is incorporated, where a locally hosted Large Language Model (Phi-3 via Ollama) generates concise and factual explanations for recommended products, enhancing transparency and user trust.

The system also includes advanced features such as multilingual query support through automatic translation, voice-based search, pagination, sorting, and an interactive user interface. Performance is evaluated using the Precision@K metric to measure retrieval effectiveness.

The proposed system demonstrates significant improvements in search relevance, scalability, and user experience by effectively combining Natural Language Processing, Computer Vision, and Information Retrieval techniques. It highlights the practical application of multimodal AI and LLM-based retrieval systems in modern e-commerce environments.

CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1	INTRODUCTION	1
	1.1. Background	2
	1.2. Objectives	3
	1.3. Purpose, Scope, and Applicability	3
	1.4. Achievements	4
	1.5. Organization of Report	7
2	SURVEY OF TECHNOLOGIES	8
3	REQUIREMENTS AND ANALYSIS	31
	3.1. Problem Definition	32
	3.2. Requirements Specification	33
	3.3. Planning and Scheduling	36
	3.4. Software and Hardware Requirements	38
	3.5. Preliminary Product Description	39
	3.6. Conceptual Models	40
4	SYSTEM DESIGN	43
	4.1. Basic Modules	44
	4.2. Data Design	46
	4.3. Procedural Design	53
	4.4. User interface design	55
5	IMPLEMENTATION AND TESTING	58
	5.1. Implementation Approaches	59
	5.2. Coding Details and Code Efficiency	61
	5.3. Testing Approach	66

6	RESULTS AND DISCUSSION	71
	6.1. Test Reports	72
	6.2 Discussion	82
7	CONCLUSIONS	84
	7.1. Conclusion	85
	7.2. Limitations of the System	86
	7.3. Future Scope of the Project	87
8	APPENDICES	91
	8.1 Appendix 1: Dataset Sample	92
	8.2 Appendix 2: Code Snippets	95
	8.3 Appendix 3: API Details	97
	REFERENCES	99

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Functional Requirements of the System	34
Table 3.2	Non-Functional Requirements of the System	35
Table 3.3	Project Development Phases	37
Table 3.4	Software Requirements	38
Table 3.5	Hardware Requirements	39
Table 4.1	Dataset Attributes Description	47
Table 4.2	Engineered Features in Dataset	47
Table 5.1	Libraries Used and Their Roles	65
Table 5.2	Testing Scenarios and Expected Outcomes	70
Table 6.1	Precision@K Evaluation Results	82
Table 8.1	Basic Product Information	92
Table 8.2	Additional Attributes	93
Table 8.3	Engineered Features	93

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Figure 3.1	Workflow of System	40
Figure 3.2	Data Flow Model	41
Figure 3.3	Data Flow Diagram	41
Figure 3.4	Use case diagram	42
Figure 4.1	Data Flow Representation	53
Figure 5.1	Retrieval Logic	63
Figure 6.1	Text-based search results for query "red shoes for men"	72
Figure 6.2	Image-based search results using uploaded product image	73
Figure 6.3	Multilingual search results for non-English query	74
Figure 6.4	Voice-based search functionality	75
Figure 6.5	Hybrid retrieval showing ranked product results	76
Figure 6.6	Best product highlighted based on ranking	77
Figure 6.7	Chatbot providing general shopping assistance	78
Figure 6.8	Product modal displaying detailed product information	79
Figure 6.9	Shopping cart displaying selected products	80
Figure 6.10	Dark mode interface of the application	81

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
CV	Computer Vision
FAISS	Facebook AI Similarity Search
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
API	Application Programming Interface
UI	User Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
CLIP	Contrastive Language-Image Pretraining
ViT	Vision Transformer
JSON	JavaScript Object Notation
ANN	Approximate Nearest Neighbor
DL	Deep Learning
TF-IDF	Term Frequency–Inverse Document Frequency
BERT	Bidirectional Encoder Representations from Transformers
SBERT	Sentence-BERT
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol

CHAPTER 1
INTRODUCTION

1.1 Background

The rapid growth of e-commerce platforms has transformed the way consumers search, evaluate, and purchase products. Modern platforms such as Amazon, Flipkart, and other online marketplaces host millions of products across diverse categories including fashion, electronics, home appliances, and accessories. This exponential growth in product catalogs has created a significant challenge in efficiently retrieving relevant items based on user queries.

Traditional search systems in e-commerce primarily rely on keyword-based matching techniques. These systems operate by comparing user queries with product titles, descriptions, and metadata. While such approaches are computationally efficient, they often fail to capture the semantic meaning and intent behind user queries. As a result, users may receive irrelevant or partially relevant results, leading to poor user experience and reduced conversion rates.

For instance, a user searching for “comfortable office shoes” may not obtain accurate results if the product listings do not explicitly contain the exact keywords “comfortable” or “office.” Similarly, queries expressed in natural language or containing synonyms may not be effectively processed by conventional systems.

In addition to textual limitations, traditional systems largely ignore visual information. However, product appearance plays a crucial role in user decision-making, especially in domains such as fashion and accessories. The inability to leverage image-based similarity further restricts the effectiveness of recommendation systems.

Recent advancements in Artificial Intelligence, particularly in Natural Language Processing (NLP) and Computer Vision, have enabled the development of intelligent retrieval systems that understand semantic meaning and visual features. Techniques such as transformer-based embeddings, multimodal learning, and vector similarity search have revolutionized information retrieval.

This project leverages these advancements to develop a Multimodal AI-Powered E-Commerce Product Recommendation System, which integrates semantic text understanding, image similarity, hybrid ranking mechanisms, and LLM-based explanation generation to provide accurate and explainable product search results.

1.2 Objectives

The primary objective of this project is to design and implement an intelligent e-commerce product search and recommendation system that improves search relevance and user experience using advanced Artificial Intelligence techniques. Traditional keyword-based systems often fail to understand user intent, resulting in less accurate results. This system addresses these limitations by incorporating semantic search, image-based retrieval, and hybrid ranking mechanisms. It also enables multimodal interaction through text, image, and voice inputs, making the system more flexible and user-friendly.

The key objectives are as follows:

1. To develop a semantic search system using transformer-based embeddings that understands user intent beyond keyword matching.
2. To implement image-based product retrieval using deep learning models for visual similarity detection.
3. To design a hybrid retrieval mechanism that combines textual and visual features for improved ranking accuracy.
4. To utilize FAISS for efficient and scalable similarity search in high-dimensional vector spaces.
5. To integrate an LLM-based explanation module that generates clear and explainable product insights using a locally deployed Large Language Model.
6. To support multilingual queries through automatic language detection and translation.
7. To develop an interactive user interface with advanced features such as voice search, sorting, pagination, and product visualization.
8. To evaluate system performance using metrics such as Precision@K.

1.3 Purpose, Scope, and Applicability

Purpose

The purpose of this project is to overcome the limitations of traditional e-commerce search systems by introducing an AI-driven multimodal recommendation framework. The system aims to improve accuracy, scalability, and explainability in product search.

Scope

The scope of the project includes:

1. Processing structured and unstructured product data
2. Generating embeddings for text and images
3. Implementing vector-based similarity search
4. Developing a hybrid ranking mechanism
5. Integrating a local LLM for explanation generation
6. Building a responsive web-based user interface

The system is designed to be scalable and can be extended to large-scale e-commerce platforms.

Applicability

The proposed system can be applied in:

1. E-commerce platforms
2. Online retail recommendation systems
3. Personalized product search engines
4. AI-based shopping assistants
5. Visual search applications

1.4 Achievements

During the development of this project, several significant milestones and technical achievements were accomplished. These contributions highlight the practical challenges encountered and the solutions implemented throughout the system development lifecycle.

1. Dataset Engineering

Multiple publicly available datasets from diverse domains such as fashion products, electronic items, and general e-commerce listings were collected and integrated. These datasets were obtained from various sources, including both structured and semi-structured public repositories.

A comprehensive data preprocessing pipeline was developed to ensure data quality and consistency. The preprocessing steps included:

1. Cleaning inconsistent and missing values

2. Standardizing attributes across different datasets
3. Merging multiple datasets into a unified and consistent format

The final dataset consists of well-structured attributes such as **product title, description, price, category, brand, color, ratings, reviews, and image paths**, which enable effective multimodal processing for both text and image-based retrieval.

2. Embedding Generation Optimization

Embedding generation was identified as one of the most computationally intensive stages of the system. Initially, embeddings were generated using CPU resources, which required approximately **5–6 hours** to process the entire dataset.

To improve efficiency and scalability:

1. GPU acceleration was adopted
2. Parallel processing capabilities were utilized

This optimization reduced the processing time significantly to **a few minutes**, demonstrating effective resource utilization and system performance improvement.

3. Text Embedding Implementation

Semantic embeddings for textual data were generated using **Sentence Transformers (BAAI/bge-small-en-v1.5)**. These embeddings enable the system to understand the contextual meaning of queries and product descriptions, thereby supporting **semantic search** instead of traditional keyword-based retrieval.

4. Image Embedding Implementation

Visual feature extraction was performed using **OpenCLIP (Vision Transformer - ViT-B/32)**. The model generates normalized image embeddings, allowing the system to retrieve products based on **visual similarity**, which is particularly important in domains such as fashion and design.

5. FAISS Index Construction

Separate FAISS indices were constructed for:

1. Text embeddings
2. Image embeddings

This enables efficient **Approximate Nearest Neighbor (ANN)** search in high-dimensional vector space, ensuring **fast and scalable retrieval performance** even for large datasets.

6. Hybrid Retrieval Algorithm

Multiple retrieval and ranking strategies were explored during system development. Initially, simple similarity-based approaches were used; however, they did not consistently produce optimal results.

To improve ranking accuracy, a **hybrid scoring mechanism** was developed, combining:

1. Semantic similarity scores
2. Keyword-based boosting
3. Attribute-based filtering (such as price, category, and color)

This significantly enhanced the relevance of search results.

7. LLM Integration and Optimization

A Large Language Model (LLM) was integrated to generate explanations for recommended products. Initially, the **Mistral model** was used; however, due to system limitations and resource constraints, it was replaced with the lightweight **Phi-3 model**.

During early stages, the model exhibited **hallucination issues**, generating inaccurate or fabricated information. To address this:

1. Prompt engineering techniques were applied
2. Strict instruction constraints were enforced

This ensured that the model generates **concise, factual, and context-aware explanations** based only on retrieved product data.

8. Frontend Development

A modern and interactive user interface was developed to enhance user experience. The frontend includes multiple advanced features:

1. **Text-based search** for semantic queries
2. **Voice search** for hands-free interaction
3. **Image-based search** for visual product retrieval
4. **Multilingual support** for diverse users
5. **Dark mode** for improved accessibility

6. **Shopping cart functionality**
7. **Pagination and sorting options**
8. **Integrated chatbot assistant** for user guidance

These features collectively improve usability and system interaction.

9. API Development

A scalable backend system was implemented using **FastAPI**, providing RESTful endpoints for:

1. Text-based product search
2. Image-based hybrid retrieval
3. Chatbot interaction
4. Product recommendation and explanation

The API ensures efficient communication between frontend and backend components, supporting real-time response generation.

1.5 Organization of the Report

The report is structured into multiple chapters to provide a comprehensive understanding of the system:

1. Chapter 1 introduces the project, objectives, scope, and achievements.
2. Chapter 2 presents the literature review of relevant technologies and research.
3. Chapter 3 discusses system requirements and problem analysis.
4. Chapter 4 explains the system design, architecture, and modules.
5. Chapter 5 describes the implementation details and algorithms.
6. Chapter 6 presents results, evaluation, and performance analysis.
7. Chapter 7 concludes the project and discusses future enhancements.
8. Chapter 8 lists references used in the project.

CHAPTER 2

SURVEY OF TECHNOLOGIES

2.1 Literature Review

1. Title: Sentence-BERT: Sentence Embeddings using Siamese BERT Networks

Author: Nils Reimers and Iryna Gurevych

Year: 2019

Sentence-BERT (SBERT) introduced a modification of the BERT architecture to generate semantically meaningful sentence embeddings using Siamese and triplet network structures. Unlike traditional BERT, which is computationally expensive for similarity comparison, SBERT enables efficient cosine similarity-based retrieval in vector space. This advancement significantly improved semantic search, clustering, and recommendation systems.

The proposed system utilizes Sentence-Transformer models to convert product titles and descriptions into dense vector embeddings, enabling meaning-based retrieval instead of traditional keyword matching.

2. Title: CLIP: Learning Transferable Visual Models from Natural Language Supervision

Author: Alec Radford et al.

Year: 2021

CLIP (Contrastive Language-Image Pretraining) introduced a multimodal learning framework that maps images and text into a shared embedding space using contrastive learning. This enables cross-modal retrieval, zero-shot classification, and visual similarity detection.

In the proposed system, OpenCLIP is used to generate image embeddings for e-commerce products. This allows visually similar product retrieval and supports multimodal recommendation by integrating textual and image features.

3. Title: FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors

Author: Jeff Johnson, Matthijs Douze, and Hervé Jégou

Year: 2017

FAISS (Facebook AI Similarity Search) is a high-performance library designed for efficient similarity search in high-dimensional vector spaces. It supports Approximate Nearest

Neighbour (ANN) indexing methods such as IVF, HNSW, and Product Quantization, enabling scalable retrieval across millions of vectors.

The proposed system employs FAISS as the core indexing engine for both text and image embeddings. Its efficient indexing ensures real-time product search and scalability for large e-commerce datasets.

4. Title: Vector Databases in Modern Applications: Real-Time Search, Recommendations, and Retrieval-Augmented Generation (RAG)

Author: Guru Pramod Rusum and Sunil Anasuri

Year: 2024

This study analyzes the role of vector databases in enabling semantic search, recommendation systems, and RAG-based pipelines. The paper discusses ANN search techniques, scalability challenges, and benchmarking across vector database systems such as FAISS, Milvus, Pinecone, and Qdrant.

The findings emphasize that vector databases are essential for real-time personalized recommendations and multimodal AI systems. This directly supports the architectural design of the proposed system, particularly the use of FAISS for scalable product retrieval and hybrid ranking.

5. Title: AI-Curated Embeddings: A Semantic Approach to Structuring and Indexing Vector Databases

Author: Arunkumar Medisetty

Year: 2023

This paper proposes a semantic-first curation framework for improving embedding quality before indexing into vector databases. By integrating transformer-based encoding, contrastive learning, clustering, and embedding refinement techniques, the study demonstrates measurable improvements in retrieval precision, clustering coherence, and latency reduction across domains including e-commerce and healthcare.

The results show a 23% improvement in top-5 precision for product search applications. These findings support the importance of high-quality embeddings in enhancing recommendation performance, which aligns with the embedding generation, and preprocessing strategy used in the proposed system.

6. Title: Retrieval-Augmented Generation for Intelligent Querying of Databases and Documents

Author: Owais Ali

Year: 2025

This paper explores the integration of Retrieval-Augmented Generation (RAG) with structured and unstructured databases to enable intelligent querying and context-aware response generation. The study highlights how vector-based retrieval systems can enhance database querying by retrieving relevant contextual information before passing it to a Large Language Model (LLM) for response generation.

The research demonstrates that combining vector search with generative models improves factual grounding, reduces hallucinations, and enhances interpretability. This directly aligns with the proposed system's RAG-based explanation module, where retrieved product information is supplied to a local LLM to generate concise and explainable recommendations.

These contributions from the theoretical and practical foundation for the proposed **Multimodal AI-Powered E-Commerce Product Recommendation System using FAISS and Hybrid Retrieval**

2.2 Introduction

With the rapid evolution of digital commerce, modern e-commerce platforms are required to handle massive volumes of heterogeneous data, including structured data such as product attributes and unstructured data such as textual descriptions and images. Traditional search systems based on keyword matching are increasingly insufficient to meet user expectations, especially when dealing with natural language queries and visual product exploration.

To address these limitations, advanced technologies from multiple domains such as Machine Learning, Natural Language Processing (NLP), Computer Vision, and Information Retrieval are integrated into intelligent systems. These technologies enable semantic understanding, multimodal processing, efficient similarity search, and explainable recommendations.

The proposed system leverages a combination of these technologies to build a multimodal product recommendation system. This chapter provides a detailed survey of the core technologies used, their working principles, advantages, and their role in the system.

2.3 Machine Learning

Machine Learning (ML) is a subset of Artificial Intelligence that focuses on building systems capable of learning patterns from data and improving performance over time without being explicitly programmed.

In traditional systems, rules are manually defined. However, in machine learning systems, patterns are learned automatically from data. This makes ML highly suitable for applications such as recommendation systems, where patterns are complex and dynamic.

Types of Machine Learning

- 1. Supervised Learning**

Uses labelled data to train models.

Example: classification, regression

- 2. Unsupervised Learning**

Finds hidden patterns in unlabeled data

Example: clustering, similarity detection

- 3. Reinforcement Learning**

Learns through interaction with environment

Role in This Project

In this system, machine learning is used for:

1. Generating embeddings (text and image)
2. Learning semantic relationships
3. Improving retrieval accuracy

Deep learning models such as transformers and vision models are used, which fall under advanced machine learning techniques.

2.4 Natural Language Processing

Natural Language Processing (NLP) is a branch of Artificial Intelligence that focuses on enabling machines to understand, interpret, and generate human language in a meaningful way. In modern applications such as chatbots, search engines, and recommendation systems, NLP plays a critical role in processing textual data.

In e-commerce systems, users interact primarily through text queries. These queries can vary widely in structure, vocabulary, and intent. Therefore, an effective NLP system must be capable of understanding both the syntax and semantics of user input.

2.4.1 Importance of NLP in E-Commerce

NLP is essential in e-commerce for several reasons:

1. Users often provide natural language queries instead of exact keywords
2. Queries may include descriptive phrases (e.g., “comfortable shoes for office use”)
3. Synonyms and variations are common (e.g., “t-shirt”, “tee”, “top”)
4. Queries may be incomplete or ambiguous

Without NLP, systems rely only on exact keyword matching, which results in poor search performance.

2.4.2 NLP Pipeline

The NLP pipeline consists of several preprocessing steps:

1. Text Cleaning

1. Removal of punctuation and special characters
2. Handling missing or null values
3. Removing unnecessary symbols

2. Tokenization

1. Splitting text into smaller units (words or tokens)
2. Example:
“Black formal shoes” → [“Black”, “formal”, “shoes”]

3. Normalization

1. Converting text to lowercase

2. Removing stopwords (optional)
3. Standardizing formats
4. Lemmatization / Stemming
 1. Reducing words to their base form
 2. Example:
“running” → “run”

2.4.3 Semantic Understanding

One of the most important aspects of NLP is semantic understanding, which refers to the ability to understand the meaning of text rather than just matching words.

Example:

1. Query: “cheap sneakers”
2. Product: “affordable sports shoes”

Even though the words are different, the meaning is similar. Semantic models can identify this relationship.

2.4.4 Application in Proposed System

In this project, NLP is used for:

1. Processing user queries
2. Cleaning and normalizing product descriptions
3. Generating semantic embeddings
4. Supporting multilingual queries through translation

NLP forms the foundation of the text-based search system, enabling accurate and context-aware retrieval.

2.5 Sentence Transformers

Sentence Transformers are a class of deep learning models specifically designed to generate dense vector representations, commonly referred to as embeddings, for sentences and paragraphs. These embeddings capture the semantic meaning of the input text, allowing efficient comparison between different textual inputs.

Traditional text representation techniques such as Bag of Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) rely on frequency-based representations. Although these methods are simple and computationally efficient, they fail to capture contextual meaning and semantic relationships between words. As a result, they are not suitable for modern applications that require deep understanding of language.

Sentence Transformers overcome these limitations by leveraging transformer-based architectures such as BERT. Unlike traditional BERT models, which require pairwise comparisons for similarity tasks, Sentence Transformers are designed to generate fixed-size embeddings for individual sentences. This makes them highly efficient for large-scale similarity search tasks.

2.5.1 Working Principle of Sentence Transformers

The working mechanism of Sentence Transformers can be understood as follows:

When a sentence is provided as input, it is first tokenized into smaller units such as words or subwords. These tokens are then passed through multiple layers of a transformer model. Each layer captures contextual relationships between words, allowing the model to understand how words influence each other within a sentence.

After processing through the transformer layers, the output is converted into a fixed-length vector representation. This vector represents the overall meaning of the sentence.

For example:

1. Sentence 1: “comfortable running shoes”
2. Sentence 2: “lightweight sports footwear”

Even though the words are different, the generated embeddings will be close to each other in vector space because their meanings are similar.

2.5.2 Cosine Similarity in Semantic Search

To compare two embeddings, cosine similarity is commonly used. Cosine similarity measures the angle between two vectors in a high-dimensional space.

If two vectors are pointing in the same direction, the cosine similarity is close to 1, indicating high similarity. If they are unrelated, the value approaches 0.

This method is highly efficient and widely used in semantic search systems.

2.5.3 Model Selection: BAAI/bge-small-en-v1.5

In this project, the Sentence Transformer model **BAAI/bge-small-en-v1.5** is used. This model is chosen because it provides a good balance between performance and computational efficiency.

The key reasons for selecting this model include:

1. Lightweight architecture suitable for local deployment
2. Faster embedding generation compared to larger models
3. High-quality semantic representations
4. Compatibility with real-time applications

2.5.4 Application in the Proposed System

In the proposed system, Sentence Transformers play a central role in enabling semantic search. Product titles, descriptions, and combined textual features are converted into embeddings. Similarly, user queries are also transformed into embeddings.

These embeddings are then compared using similarity measures to retrieve relevant products. This approach ensures that even if the exact keywords are not present, semantically related products can still be retrieved.

Thus, Sentence Transformers significantly improve the accuracy and relevance of the search system.

2.6 Computer Vision

Computer Vision is a field of Artificial Intelligence that focuses on enabling machines to interpret and understand visual information from images and videos. In e-commerce applications, visual information plays a critical role because users often rely on the appearance of products when making purchasing decisions.

Traditional search systems rely heavily on textual descriptions, which may not fully capture the visual characteristics of a product. For example, two products may have similar appearances but different textual descriptions, making it difficult for keyword-based systems to identify them as similar.

Computer Vision addresses this limitation by extracting meaningful features directly from images.

2.6.1 Importance of Visual Information in E-Commerce

Visual data provides additional context that cannot be captured through text alone. In domains such as fashion, accessories, and home decor, product appearance is often more important than textual attributes.

Users may want to:

1. Find products similar to a given image
2. Compare colors, patterns, and designs
3. Discover visually appealing alternatives

Therefore, integrating Computer Vision into e-commerce systems enhances the overall user experience.

2.6.2 Image Processing Pipeline

The process of handling images involves several steps:

1. **Image Loading**

The image is loaded into memory from a file or URL

2. **Preprocessing**

Images are resized and normalized to match model requirements

3. **Feature Extraction**

Important visual features such as color, texture, and shape are extracted

4. **Embedding Generation**

The extracted features are converted into numerical vectors

2.6.3 Feature Representation

Instead of storing raw images, systems store compact representations called embeddings. These embeddings capture the essential characteristics of the image while reducing storage and computational requirements.

2.6.4 Application in the Proposed System

In this project, Computer Vision is used to enable image-based product search. Users can upload an image, and the system retrieves visually similar products.

This feature significantly enhances usability, especially for users who may not know the exact name of a product but can identify it visually.

2.7 CLIP Model

CLIP (Contrastive Language-Image Pretraining) is a powerful multimodal model developed to understand the relationship between images and text. It is trained on a large dataset of image-text pairs and learns to associate visual and textual information in a shared embedding space.

2.7.1 Concept of Multimodal Learning

Multimodal learning refers to the integration of multiple types of data, such as text and images, into a unified model. Unlike traditional models that process only one type of data, multimodal models can understand relationships across different data formats.

CLIP is one such model that bridges the gap between visual and textual understanding.

2.7.2 Working Mechanism of CLIP

CLIP consists of two main components:

1. **Image Encoder**
Processes images and extracts visual features
2. **Text Encoder**
Processes text and extracts semantic features

Both encoders map their inputs into a shared vector space. During training, the model learns to bring matching image-text pairs closer together while pushing unrelated pairs farther apart.

2.7.3 Architecture

The architecture of CLIP includes:

1. Vision Transformer (ViT) for image processing
2. Transformer-based model for text processing

These components work together to create aligned embeddings.

2.7.4 Model Used in This Project

In this project, **OpenCLIP (ViT-B/32)** is used. It is an open-source implementation of CLIP and provides efficient performance for image embedding generation.

2.7.5 Application in the Proposed System

CLIP is used to generate embeddings for product images. These embeddings are stored in a FAISS index for efficient retrieval.

When a user uploads an image, the system:

1. Converts the image into an embedding
2. Searches for similar embeddings in the index
3. Retrieves visually similar products

2.7.6 Example Use Case

If a user uploads an image of a black sneaker, the system retrieves similar sneakers based on shape, color, and design, even if the product titles are different.

2.8 FAISS

FAISS (Facebook AI Similarity Search) is a high-performance library developed by Facebook AI Research for efficient similarity search and clustering of high-dimensional vectors. In modern AI systems, especially those involving embeddings, data is represented as numerical vectors. Searching through these vectors using traditional methods becomes computationally expensive as the dataset grows.

FAISS addresses this problem by providing optimized algorithms and data structures for fast nearest neighbor search. It is widely used in applications such as recommendation systems, image retrieval, and semantic search.

2.8.1 Need for FAISS

In this project, both textual and visual data are converted into embeddings. These embeddings are high-dimensional vectors, often containing hundreds of numerical values.

If a dataset contains thousands or millions of such vectors, performing a simple linear search to find similar vectors becomes extremely slow. For example:

1. Comparing one query vector with 100,000 vectors → very high computation
2. Real-time systems require results within milliseconds

FAISS solves this problem by organizing vectors into efficient index structures, enabling fast similarity search.

2.8.2 Working Principle of FAISS

The working of FAISS can be summarized in the following steps:

1. **Embedding Storage**

All product embeddings (text and image) are stored in a FAISS index

2. **Query Processing**

When a user enters a query, it is converted into an embedding

3. **Similarity Comparison**

The query embedding is compared with stored vectors

4. **Nearest Neighbor Retrieval**

The system retrieves the most similar vectors based on distance or similarity score

2.8.3 Similarity Metrics

FAISS supports multiple similarity metrics, including:

1. L2 Distance (Euclidean Distance)
2. Inner Product (Dot Product)
3. Cosine Similarity

In this project, cosine similarity (via normalized vectors) is used because it effectively measures semantic similarity.

2.8.4 Types of FAISS Indexes

FAISS provides different types of indexes depending on the use case:

1. **Flat Index**

Performs exact search

High accuracy but slower for large datasets

1. **IVF (Inverted File Index)**

Divides data into clusters

Faster search with slight approximation

2. **HNSW (Hierarchical Navigable Small World)**

Graph-based search

Very fast and scalable

In this project, an **IVF (Inverted File Index)** is used to improve search efficiency.

2.8.5 Application in the Proposed System

FAISS is used as the core retrieval engine in the system. Two separate indexes are created:

1. **Text FAISS Index** → for semantic search
2. **Image FAISS Index** → for visual similarity search

When a query is processed:

1. The system retrieves top results from FAISS
2. These results are further refined using hybrid ranking

This ensures both **speed and accuracy** in retrieval.

2.9 Hybrid Retrieval System

Hybrid retrieval is a technique that combines multiple retrieval strategies to improve the accuracy and relevance of search results. In e-commerce systems, relying on a single method often leads to suboptimal results.

2.9.1 Need for Hybrid Retrieval

Single-method approaches have limitations:

1. **Keyword-based search**
 - Cannot understand context
 - Fails with synonyms
2. **Semantic search (embeddings)**
 - May ignore exact keyword importance
 - Can sometimes return loosely related results

To overcome these issues, a hybrid approach is used.

2.9.2 Working of Hybrid Retrieval

The hybrid retrieval system combines multiple scores to compute a final ranking score.

The general formula is:

Final Score = (Weight₁ × Semantic Similarity) + (Weight₂ × Keyword Matching Score) + (Optional Image Similarity Score)

2.9.3 Components of Hybrid Ranking

1. Semantic Similarity Score

1. Computed using embeddings
2. Measures meaning similarity

2. Keyword Matching Score

1. Checks presence of keywords
2. Boosts exact matches

3. Image Similarity Score

1. Used in image-based search
2. Compares visual features

2.9.4 Application in Proposed System

In this project, multiple ranking strategies were experimented with before finalizing a hybrid approach. The final system combines:

1. Semantic similarity (primary factor)
2. Keyword boosting
3. Filtering based on attributes (price, category, color)

This significantly improves search relevance and user satisfaction.

2.10 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an advanced technique that combines information retrieval with natural language generation. It addresses one of the major limitations of Large Language Models (LLMs), which is hallucination.

2.10.1 Problem with Traditional LLMs

LLMs generate responses based on learned knowledge. However:

1. They may generate incorrect information
2. They lack real-time data access
3. They may hallucinate facts

2.10.2 Concept of RAG

RAG improves LLM performance by providing external knowledge.

Process:

1. Retrieve relevant data from database
2. Pass retrieved data to LLM
3. Generate response based only on provided data

2.10.3 Benefits of RAG

1. Improves factual accuracy
2. Reduces hallucination
3. Provides explainable outputs
4. Combines retrieval + generation

2.10.4 Application in Proposed System

In this project, concepts similar to Retrieval-Augmented Generation (RAG) are considered for improving explanation quality. However, a fully integrated RAG pipeline is not implemented.

The system uses a combination of retrieval mechanisms and a locally deployed Large Language Model (LLM) to provide explanations for products. The retrieval module identifies relevant products using FAISS, and the LLM generates general explanations based on product information.

Workflow:

1. User query is processed
2. Relevant products are retrieved using FAISS
3. Product details are displayed to the user
4. The LLM generates context-aware explanations using the retrieved product data

2.11 Large Language Models (LLMs)

Large Language Models (LLMs) are advanced AI models trained on vast amounts of text data to understand and generate human-like language.

2.11.1 Working of LLMs

LLMs are based on transformer architecture and use:

1. Self-attention mechanisms
2. Contextual understanding
3. Sequential prediction

They predict the next word based on previous words.

2.11.2 Models Used in Project

1. **Initial Model:** Mistral
2. **Final Model:** Phi-3

2.11.3 Challenges Faced

During development, several challenges were encountered:

1. High memory usage
2. Slow performance
3. Hallucination (incorrect responses)

2.11.4 Solutions Implemented

To overcome these issues:

1. Switched to lightweight Phi-3 model
2. Applied prompt engineering
3. Restricted model output to product data

2.11.5 Application in Proposed System

LLMs are used to:

1. Generate product explanations
2. Improve user interaction
3. Provide intelligent responses

2.12 FastAPI

FastAPI is a modern, high-performance web framework used for building APIs in Python.

2.12.1 Features of FastAPI

1. High speed (based on ASGI)
2. Easy to develop
3. Automatic documentation
4. Supports asynchronous programming

2.12.2 Role in Proposed System

FastAPI is used as the backend framework to handle:

1. Search queries
2. Image-based retrieval
3. Chatbot interaction

2.12.3 API Endpoints

Examples:

1. /search → text search
2. /image-search → image retrieval
3. /chat → chatbot

2.13 Multilingual Processing

Multilingual processing is an essential component of modern intelligent systems, especially in applications such as e-commerce platforms that serve a diverse and geographically distributed user base. Users often prefer to interact with systems in their native or regional languages rather than a single dominant language such as English. Therefore, incorporating multilingual capabilities significantly enhances accessibility, usability, and user engagement.

In traditional systems, users are required to input queries in a specific language, typically English. This creates a barrier for users who are more comfortable using regional languages. To overcome this limitation, the proposed system integrates a multilingual processing mechanism that allows users to search for products using different languages seamlessly.

2.13.1 Need for Multilingual Processing

The need for multilingual support arises due to several practical considerations. Firstly, e-commerce platforms cater to a **diverse user base** spanning different regions, cultures, and linguistic backgrounds. In countries like India, users commonly communicate in languages such as Hindi, Telugu, Tamil, and many others. Restricting the system to a single language would limit its usability.

Secondly, users often prefer to search in their **native or regional language**, as it is more natural and convenient. For example, a user may search for “काले जूते” (black shoes in Hindi) instead of typing the same query in English.

Thirdly, multilingual support enhances **user inclusivity and accessibility**, making the system more user-friendly for individuals who may not be proficient in English. This is especially important for reaching a broader audience and improving user satisfaction.

2.13.2 Working Mechanism of Multilingual Processing

The multilingual processing pipeline in the proposed system consists of multiple stages that ensure accurate and efficient handling of user queries across different languages.

1. Language Detection

When a user enters a query, the system first identifies the language of the input text using a language detection mechanism. This step is crucial because it determines how the query will be processed further.

The detection process is automatic and does not require any manual selection from the user. This makes the system more intuitive and user-friendly.

2. Translation Layer (Frontend Integration)

After detecting the language, the query is passed through a **translation layer implemented in the frontend**. This layer converts the input query into a common processing language, which in this system is English.

The translation layer ensures that:

1. Queries from different languages are normalized
2. The backend system can process all queries uniformly
3. Semantic meaning of the query is preserved

3. Query Processing

Once translated into English, the query undergoes the standard processing pipeline:

1. Text cleaning and normalization
2. Conversion into embeddings using Sentence Transformers
3. Retrieval using FAISS

Since all queries are converted into a single language, the system maintains consistency in processing and avoids complexity in handling multiple languages separately.

4. Result Retrieval and Display

After processing, the system retrieves relevant products based on semantic similarity and hybrid ranking. The results are then displayed to the user through the interface.

Although the backend operates in English, the user experience remains seamless because the input can be provided in any supported language.

2.13.3 Application in the Proposed System

The proposed system provides multilingual capabilities that allow users to interact with the platform in multiple languages.

The user interface includes selectable options for commonly used languages such as:

1. English
2. Hindi
3. Telugu
4. Tamil

However, the system is not limited to these languages. It is capable of **automatically detecting and processing queries in a wide range of languages**, thanks to the integrated language detection and translation mechanism.

This means that even if a user enters a query in a language that is not explicitly listed in the interface, the system can still process it effectively.

2.14 Frontend Technologies

The frontend of the proposed system plays a crucial role in enabling effective interaction between the user and the underlying AI-powered recommendation engine. It serves as the interface through which users input queries, view search results, and interact with various functionalities such as voice search, image-based retrieval, and multilingual processing.

The frontend is designed to be intuitive, responsive, and user-friendly, ensuring a seamless experience for users with different levels of technical expertise. It integrates multiple advanced features that enhance usability, accessibility, and overall system performance.

2.14.1 Technologies Used

The frontend is developed using standard web technologies, which provide flexibility, compatibility, and ease of integration with backend APIs.

1. HTML (Hypertext Markup Language)

HTML is used to structure the content of the web interface. It defines the layout of elements such as input fields, buttons, product cards, and navigation components. HTML provides the foundation for building the user interface.

2. CSS (Cascading Style Sheets)

CSS is used to design and style the frontend interface. It controls aspects such as layout, colors, fonts, spacing, and responsiveness. The inclusion of features such as dark mode is achieved through CSS styling, enhancing visual comfort for users.

3. JavaScript

JavaScript is used to add interactivity and dynamic behavior to the frontend. It handles user actions such as search queries, voice input, image uploads, and API requests. JavaScript also enables real-time updates of search results without requiring full page reloads.

2.14.2 Features Implemented

The frontend includes a wide range of features designed to improve user interaction and system functionality.

1. Text-Based Search

The system supports semantic text-based search, allowing users to enter queries in natural language. The input is processed and sent to the backend, where it is converted into embeddings for similarity-based retrieval. This enables users to find relevant products even when exact keywords are not used.

2. Voice Search

A voice search feature is implemented to allow users to interact with the system using speech input. The spoken query is converted into text using speech recognition techniques and then processed by the system. This feature enhances accessibility and provides a convenient hands-free search option.

3. Image-Based Search

The system supports image-based search, where users can upload an image of a product. The uploaded image is processed using a deep learning model to extract visual features. The system then retrieves visually similar products based on image embeddings. This is particularly useful when users are unable to describe a product in words.

4. Shopping Cart Functionality

A shopping cart feature is implemented to simulate real-world e-commerce functionality. Users can add selected products to the cart for later viewing or purchase. This feature enhances usability and provides a more realistic shopping experience.

5. Chatbot Integration

A chatbot is integrated into the frontend to assist users in product discovery and query resolution. The system uses a Retrieval-Augmented Generation (RAG) approach, where relevant products are retrieved using FAISS and passed to the LLM to generate accurate, context-aware responses. This improves recommendation quality and enhances user experience.

6. Multilingual Support

The frontend includes multilingual support, allowing users to interact with the system in different languages. A translation layer processes the input and converts it into a standard

format for backend processing. This feature makes the system accessible to users from diverse linguistic backgrounds.

7. Relevance-Based Sorting

The system includes a relevance-based sorting mechanism that ranks products based on their similarity to the user query. The ranking is determined using a hybrid scoring system that combines semantic similarity and keyword matching. This ensures that the most relevant products appear at the top of the results.

8. Pagination and Sorting Options

To efficiently handle large numbers of search results, pagination is implemented. Users can navigate through multiple pages of results. Additionally, sorting options such as price-based sorting (low to high or high to low) are provided, giving users greater control over how results are displayed.

9. Dark Mode

A dark mode feature is implemented to enhance visual comfort, especially in low-light conditions. It also provides a modern and aesthetically pleasing interface, improving overall user experience.

2.14.3 Importance of Frontend Technologies

The frontend plays a critical role in the success of the system. It directly influences how users interact with the platform and perceive its effectiveness.

The importance of the frontend can be summarized as follows:

1. It improves usability by providing an intuitive interface
2. It enhances user experience through interactive features
3. It enables seamless communication between the user and backend system
4. It supports advanced functionalities such as voice and image search
5. It ensures accessibility through multilingual support

A well-designed frontend is essential for translating complex backend processes into a simple and user-friendly experience.

CHAPTER 3

REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

The rapid growth of e-commerce platforms has led to the availability of massive product catalogs consisting of both structured and unstructured data. These datasets include attributes such as product titles, descriptions, categories, prices, brands, and images. While the availability of such data provides opportunities for enhanced product discovery, it also introduces significant challenges in retrieving relevant products efficiently.

Traditional e-commerce search systems rely primarily on keyword-based matching techniques. These systems compare user queries with product metadata and return results based on exact or partial keyword matches. Although such systems are efficient and scalable, they fail to capture the semantic meaning and intent behind user queries.

For instance, when a user searches for “comfortable office shoes,” the system may fail to retrieve relevant products if the product descriptions contain terms such as “formal footwear” or “soft sole shoes” instead of the exact keywords used in the query. This limitation results in poor search accuracy and reduced user satisfaction.

Another major limitation of traditional systems is the lack of support for visual similarity search. In many e-commerce domains, particularly fashion and accessories, users rely heavily on visual appearance when making purchasing decisions. However, most systems do not utilize image data for retrieval, thereby missing an important dimension of product similarity.

Furthermore, traditional systems lack explainability. Users are presented with search results without any reasoning or explanation as to why a particular product was recommended. This reduces user trust and transparency.

To overcome these limitations, there is a need for an intelligent system that integrates semantic understanding, visual processing, efficient retrieval mechanisms, and explainable AI. The proposed system addresses these challenges by combining Natural Language Processing, Computer Vision, FAISS-based similarity search, hybrid ranking, and Retrieval-Augmented Generation.

Key Problems:

- 1. Lack of Semantic Understanding**

Systems fail to understand the actual meaning of queries.

Example:

“comfortable office shoes” may not match “formal soft footwear”

2. **Synonym and Variation Issues**

Different words with similar meaning are not recognized

3. **No Visual Search Capability**

Traditional systems ignore image-based similarity

4. **Poor Ranking Mechanism**

Results are not always relevant or properly ordered

5. **Lack of Explainability**

Users do not know why a product is recommended

Problem Statement

There is a need to develop an **intelligent e-commerce system** that:

1. Understands user intent using semantic processing
2. Supports image-based product search
3. Provides accurate and relevant results
4. Generates explainable outputs
5. Handles multilingual queries

3.2 Requirements Specification

Requirements specification defines the functionalities and quality attributes that the system must satisfy. It serves as a blueprint for system development and ensures that all components are aligned with user needs.

3.2.1 Functional Requirements

Functional requirements describe the operations that the system must perform.

Requirement	Description
Text-Based Search	The system should allow users to enter natural language queries and retrieve relevant products using semantic search
Image-Based Search	The system should allow users to upload an image and retrieve visually similar products

Voice Search	The system should support speech input and convert it into text for processing
Multilingual Support	The system should detect and translate queries from multiple languages
Product Recommendation	The system should recommend relevant products based on hybrid retrieval
Chatbot Interaction	The system should provide chatbot assistance using an LLM
Filtering Options	The system should allow filtering based on attributes such as price and category
Sorting Mechanism	The system should provide relevance-based and price-based sorting
Cart Management	The system should allow users to add and manage products in a shopping cart

Table 1

Table 3.1: Functional Requirements of the System

Explanation of Functional Requirements

The system is designed to provide a comprehensive and interactive user experience. The **text-based search** functionality enables users to enter queries in natural language, which are processed using semantic embeddings. This ensures that the system understands the meaning of the query rather than relying solely on keyword matching. Uses **Sentence Transformers** to understand query meaning instead of exact keywords.

The **image-based search** functionality allows users to upload product images and retrieve visually similar items using deep learning models. This feature is particularly useful in scenarios where users are unable to describe products in words. Uses **OpenCLIP** to match visual similarity.

Voice search enhances accessibility by allowing users to interact with the system using speech input. The integration of multilingual support ensures that users can search in their preferred language, with automatic translation handled by the system. Converts speech to text and processes normally.

The chatbot functionality provides intelligent assistance to users by responding to shopping-related queries, offering suggestions, and helping users understand product categories. It is integrated with the product retrieval system using a Retrieval-Augmented Generation (RAG) approach, where relevant products are retrieved and passed to the LLM to generate context-aware responses, ensuring more accurate and helpful recommendations.

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

Requirement	Description
Performance	The system should provide fast response times for search queries
Scalability	The system should handle large datasets efficiently
Accuracy	The system should return highly relevant search results
Usability	The interface should be intuitive and user-friendly
Reliability	The system should operate consistently without failures
Maintainability	The system should be easy to update and extend
Security	The system should ensure safe handling of data

Table 2

Table 3.2: Non-Functional Requirements of the System

Explanation of Non-Functional Requirements

Performance is a critical requirement for any real-time system. The use of FAISS ensures that similarity search is performed efficiently, enabling quick retrieval of results. Scalability is achieved by designing the system to handle increasing amounts of data through efficient indexing mechanisms.

Accuracy is ensured through the use of semantic embeddings and hybrid ranking, which improve the relevance of search results. Usability is enhanced through a well-designed frontend interface that supports multiple interaction modes.

Reliability and maintainability are achieved through modular system design, allowing individual components to be updated independently. Security considerations ensure that user data and system integrity are maintained.

3.2.3 System Constraints

The system operates under certain constraints:

1. Limited GPU availability during development
2. Moderate dataset size (not production scale)
3. Local deployment environment
4. Lightweight LLM (Phi-3) due to hardware limits

These constraints influenced design decisions.

3.3 Planning and Scheduling

Planning and scheduling are essential for managing the development process and ensuring timely completion of the project. The project was executed in multiple phases, each focusing on a specific aspect of system development.

3.3.1 Development Approach

An iterative development approach was followed:

1. Build small modules
2. Test individually
3. Integrate gradually

where each component of the system was developed, tested, and refined before moving to the next stage. This approach allowed continuous improvement and helped in identifying issues early in the development process.

3.3.2 Project Phases

Phase	Description
Data Collection	Collecting multiple datasets from different sources
Data Preprocessing	Cleaning and merging datasets into a unified format

Embedding Generation	Generating text and image embeddings
Index Construction	Building FAISS indexes for efficient retrieval
Model Integration	Integrating LLM for explanation generation
Backend Development	Developing APIs using FastAPI
Frontend Development	Building user interface
Testing and Evaluation	Evaluating system performance

Table 3

Table 3.3: Project Development Phases

3.3.3 Practical Challenges in Scheduling

During development, several challenges were encountered:

1. Dataset integration required significant time due to inconsistencies
2. Embedding generation initially took 5–6 hours using CPU
3. GPU optimization was required to improve performance
4. Multiple ranking strategies were tested before finalizing hybrid retrieval
5. Model hallucination

These challenges were addressed through careful planning and iterative refinement.

Solutions:

1. Data cleaning pipeline
2. GPU acceleration
3. Hybrid ranking
4. Prompt engineering

3.4 Software and Hardware Requirements

3.4.1 Software Requirements

Software	Purpose
Python	Core programming language
FastAPI	Backend API development
Sentence Transformers	Text embedding generation
OpenCLIP	Image embedding generation
FAISS	Similarity search
PyTorch	Deep learning framework
Pandas	Data preprocessing
NumPy	Numerical operations
Langdetect	Language detection
Deep Translator	Translation

Table 4

Table 3.4: Software Requirements

Explanation

Each software component plays a specific role in the system. Python serves as the primary development language due to its extensive support for AI libraries. FastAPI is used for building efficient APIs, while Sentence Transformers and OpenCLIP are used for generating embeddings.

FAISS enables fast similarity search, and PyTorch provides the underlying deep learning framework. Pandas and NumPy are used for data processing and numerical computations.

3.4.2 Hardware Requirements

Hardware	Specification
Processor	Intel i5 / Ryzen 5 or higher
RAM	Minimum 8GB (16GB recommended)
Storage	Minimum 10GB
GPU	Optional (used for faster embedding generation)

Table 5

Table 3.5: Hardware Requirements

Explanation

Embedding generation is computationally intensive, especially for large datasets. Initially, embeddings were generated using CPU, which resulted in long processing times. The use of GPU significantly reduced processing time, demonstrating the importance of hardware optimization.

3.5 Preliminary Product Description

The proposed system is a **Multimodal AI-Powered E-Commerce Product Recommendation System** designed to improve the accuracy and efficiency of product search.

The system allows users to interact using text, voice, and images. It processes these inputs using advanced AI techniques and retrieves relevant products based on semantic and visual similarity.

System Workflow

The system workflow begins when a user enters a query through the frontend interface. The query may be in the form of text, voice, or image. If the input is voice-based, it is converted into text. If the input is in a regional language, it is translated into English.

The processed query is then converted into embeddings using Sentence Transformers. If the query involves an image, image embeddings are generated using OpenCLIP.

These embeddings are passed to FAISS, which retrieves similar products. A hybrid ranking mechanism refines the results, and the final products are displayed to the user. Additionally, the LLM generates explanations for the recommended products.

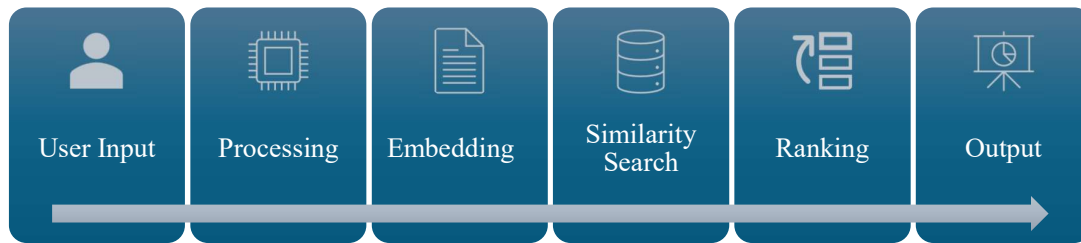


Figure 3.1 Workflow of system

Input Types:

1. Text
2. Voice
3. Image

Output:

1. Product list
2. Best product highlight
3. UI display

Key Features:

1. Semantic search
2. Image similarity
3. Multilingual support
4. Chatbot assistance

3.6 Conceptual Models

Conceptual models provide a high-level representation of the system.

3.6.1 System Architecture

The system consists of:

1. Frontend (User Interface)
2. Backend (FastAPI)

3. Embedding Models
4. FAISS Index
5. LLM Module

3.6.2 Data Flow Model

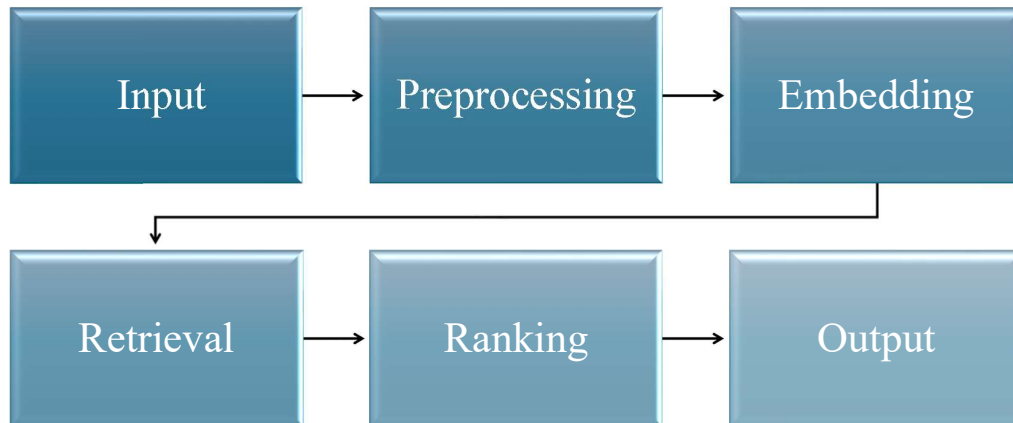


Figure 3.2 Data Flow Model

3.6.3 Data Flow Diagram

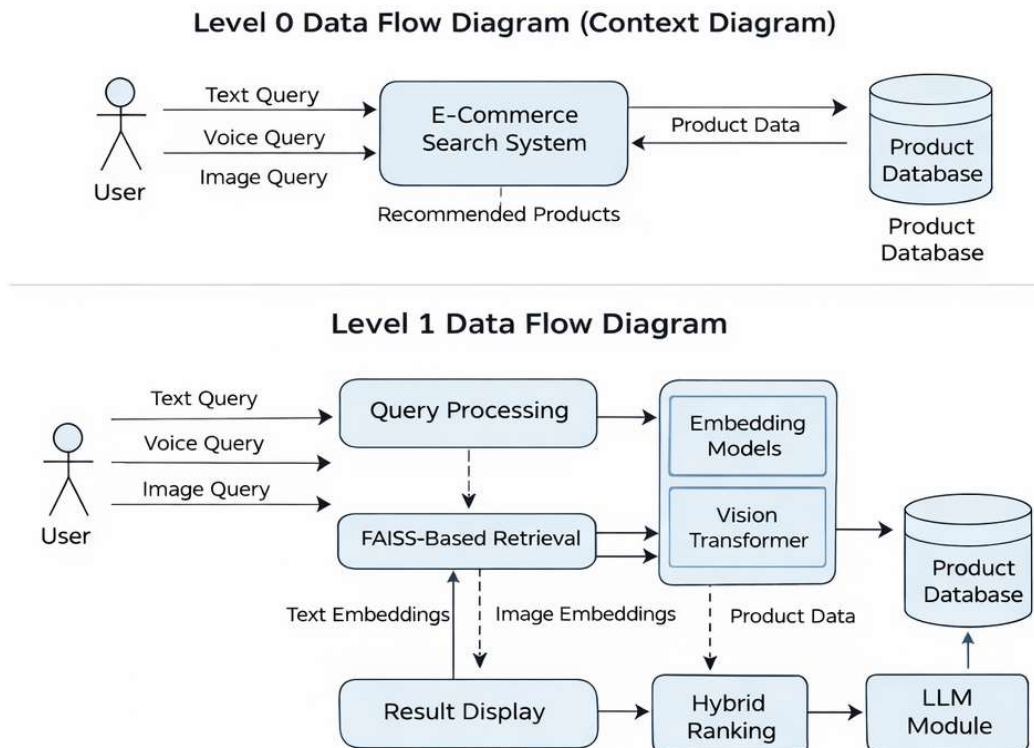


Figure 3.3 Data Flow Diagram

3.6.4 Module Description

The system is divided into multiple modules:

1. Text Retrieval Module
2. Image Retrieval Module
3. Hybrid Ranking Module
4. LLM Explanation Module
5. User Interface Module

Each module performs a specific function, ensuring modularity and scalability.

3.6.5 Use Case Diagram

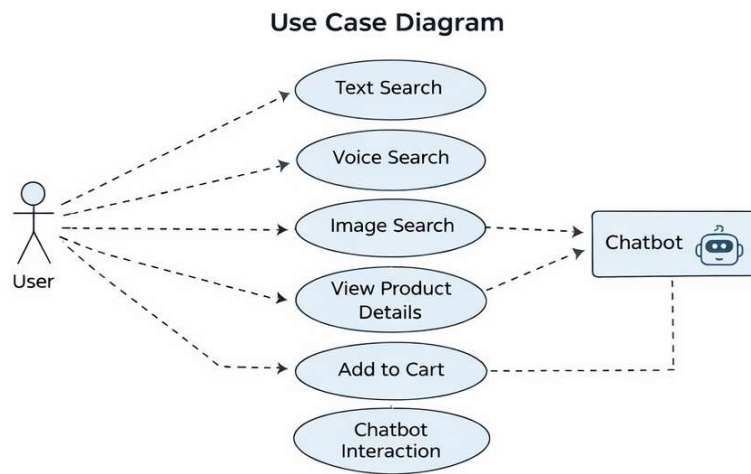


Figure 3.4 Use case diagram

CHAPTER 4

SYSTEM DESIGN

4.1 Basic Modules

The proposed system is designed using a modular architecture, where each module is responsible for a specific functionality. This modular design improves system maintainability, scalability, and flexibility.

The system is divided into several key modules, each contributing to the overall functionality of the multimodal recommendation system.

4.1.1 Text Retrieval Module

The Text Retrieval Module is responsible for processing user queries in textual form and retrieving relevant products based on semantic similarity.

When a user enters a text query, the system first preprocesses the input by cleaning and normalizing the text. The processed query is then passed to a Sentence Transformer model, which converts the text into a dense vector representation known as an embedding.

This embedding captures the semantic meaning of the query. The embedding is then compared with precomputed product embeddings stored in a FAISS index. The system retrieves the most similar products based on cosine similarity.

This module plays a crucial role in enabling semantic search, allowing the system to understand user intent even when exact keywords are not used.

4.1.2 Image Retrieval Module

The Image Retrieval Module allows users to search for products using images. This module is particularly useful in scenarios where users may not know the exact name of a product but can identify it visually.

When a user uploads an image, it is processed using the OpenCLIP model. The model extracts visual features from the image and converts them into an embedding.

This embedding is then compared with image embeddings stored in the FAISS index. The system retrieves visually similar products based on similarity scores. The integration of this module enhances the system by incorporating visual information into the retrieval process.

4.1.3 Hybrid Ranking Module

The Hybrid Ranking Module is one of the most important components of the system. It combines multiple scoring mechanisms to improve the accuracy and relevance of search results.

The module computes a final ranking score by combining:

1. Semantic similarity score
2. Keyword matching score
3. (Optional) Image similarity score

This approach ensures that both contextual meaning and exact keyword matches are considered during ranking.

The hybrid ranking mechanism was developed after experimenting with different approaches, making it a key contribution of the project.

4.1.4 LLM Explanation Module

The LLM Explanation Module is responsible for generating explanations for recommended products. After retrieving the top products, relevant product details are passed to a Large Language Model (Phi-3). The model generates concise explanations describing why a particular product matches the user query.

This module follows a Retrieval-Augmented Generation (RAG)-inspired approach, ensuring that the generated responses are grounded in actual product data.

4.1.5 Multilingual Processing Module

This module enables users to interact with the system in multiple languages. When a user enters a query in a non-English language, the system detects the language and translates it into English using a frontend translation layer. The translated query is then processed using the standard pipeline.

This module ensures accessibility and improves usability for users from different linguistic backgrounds.

4.1.6 Voice Search Module

The Voice Search Module allows users to input queries using speech. The spoken input is converted into text using speech recognition techniques.

The converted text is then processed by the system in the same way as a text query. This feature enhances accessibility and provides a more interactive user experience.

4.1.7 Frontend Module

The Frontend Module provides the user interface for interacting with the system. It includes features such as text search, image upload, voice input, multilingual selection, dark mode, and shopping cart functionality.

The frontend communicates with the backend through API calls and displays the retrieved results in a structured and user-friendly format.

4.1.8 Backend API Module

The Backend Module is implemented using FastAPI and handles all processing logic.

It provides endpoints for:

1. Text search
2. Image-based retrieval
3. Chatbot interaction

The backend acts as a bridge between the frontend and the AI models, ensuring efficient communication and data processing.

4.2 Data Design

Data design plays a crucial role in the proposed system, as it defines how product data is structured, stored, processed, and utilized for efficient retrieval. Since the system relies heavily on both textual and visual data, it is essential to design a data model that supports multimodal processing.

The dataset used in this project was constructed by integrating multiple publicly available e-commerce datasets from different domains such as fashion, electronics, and general retail. These datasets varied in structure and required significant preprocessing to ensure consistency.

4.2.1 Dataset Structure

The final dataset is organized in a structured format, where each row represents a product and each column represents a specific attribute.

The dataset includes the following key attributes:

Attribute Name	Description
product_id	Unique identifier assigned to each product
title	Name of the product
description	Detailed textual description of the product
category	Original category of the product
brand	Brand name of the product
price	Cost of the product
gender	Target audience (Men, Women, Unisex)
color	Color of the product
image_path	File path or URL of the product image
rating	Average user rating of the product
reviews	Number of user reviews

Table 6

Table 4.1: Dataset Attributes Description

In addition to these attributes, several engineered features were created to enhance system performance:

Feature Name	Description
search_text	Combined field of title, description, category, and tags used for embedding generation
category_clean	Standardized category values for consistency
price_range	Categorized price groups (e.g., Low, Medium, High)
tags	Additional keywords extracted for improving search relevance

Table 7

Table 4.2: Engineered Features in Dataset

These features ensure that the dataset is suitable for both semantic search and filtering operations.

4.2.2 Data Preprocessing and Transformation

Before using the dataset, several preprocessing steps were performed to improve data quality and consistency.

The preprocessing process included:

1. Removing duplicate entries
2. Handling missing values
3. Standardizing column formats
4. Cleaning textual data (lowercasing, removing noise)
5. Converting prices into numeric format

One of the most important steps was the creation of the **search_text** field, which combines multiple textual attributes such as title, description, category, and tags. This unified field improves the quality of semantic embeddings.

The dataset preparation phase required significant effort due to inconsistencies across different sources, making it one of the most time-consuming stages of the project.

4.2.3 Embedding Generation and Storage

The system uses two types of embeddings:

1. Text Embeddings:

Text embeddings are generated using **Sentence Transformer models** to capture the semantic meaning of product titles, descriptions, and user queries. These embeddings convert textual data into dense vector representations.

Initially, the system utilized the **all-MiniLM-L6-v2** model, which provides a good balance between performance and accuracy for semantic search tasks. However, to further optimize the system for faster response times and lower computational overhead, the model was later replaced with **BAAI/bge-small-en-v1.5**.

The **BGE (BAAI General Embedding)** model offers improved efficiency with comparable semantic understanding, making it more suitable for real-time applications such as interactive

product search and chatbot systems. Despite being lightweight, it maintains strong retrieval performance, ensuring that relevant products are accurately matched to user queries.

2. Image Embeddings:

Generated using OpenCLIP, these embeddings capture visual features of product images.

Initially, embedding generation was performed using CPU, which required approximately 5–6 hours. To optimize performance, GPU acceleration was used, reducing the processing time significantly.

The embeddings are stored in serialized format (e.g., .pkl files) and are later indexed using FAISS for efficient retrieval.

4.2.4 FAISS Index Storage

FAISS plays a critical role in the system by enabling efficient similarity search over high-dimensional embeddings. Since both textual and visual data are converted into vector representations, it becomes necessary to store and search these vectors in an optimized manner.

In the proposed system, two separate FAISS indexes are constructed:

1. Text FAISS Index, which stores embeddings generated from product textual data
2. Image FAISS Index, which stores embeddings generated from product images

The separation of these indexes allows independent processing of textual and visual queries, improving flexibility and performance.

1. Index Creation Process

The index creation process begins after generating embeddings for all products. These embeddings are stored as numerical vectors in a matrix format. Each row in the matrix corresponds to a product embedding.

A FAISS index structure (such as IndexFlatL2 or cosine-based index) is initialized, and all embeddings are added to the index. This process organizes the data in a way that allows efficient nearest neighbor search.

2. Storage of Index Files

Once the index is created, it is stored on disk for later use. The FAISS indexes are saved in binary format (e.g., .bin files), while additional metadata such as product mappings are stored in serialized formats like .pkl files.

For example:

1. Text index → faiss_index.bin
2. Image index → image_faiss_index.bin

These files are loaded during runtime to perform similarity search without recomputing embeddings.

3. Mapping Between Index and Products

FAISS stores only embeddings and does not inherently store product details. Therefore, a mapping mechanism is maintained to associate each embedding with its corresponding product.

This is typically done using:

1. A list or dictionary mapping index positions to product IDs
2. DataFrame indexing

When FAISS returns the indices of nearest neighbors, the system uses this mapping to retrieve the actual product information from the dataset.

4. Search Process Using FAISS

When a user query is processed, it is converted into an embedding. This embedding is passed to the FAISS index, which computes similarity scores between the query vector and stored vectors.

The index returns the top K most similar vectors along with their indices. These indices are then mapped back to product records.

This process is extremely fast compared to traditional search methods, enabling real-time retrieval.

5. Performance Benefits

The use of FAISS significantly improves system performance:

1. Reduces search time from linear complexity to optimized search
2. Enables handling of large datasets
3. Supports real-time recommendations

4.2.5 Data Flow Architecture

The data flow architecture defines how data moves through different components of the system, from user input to final output. In the proposed system, the data flow is designed to efficiently handle multimodal inputs such as text, voice, and images, ensuring accurate and fast retrieval of relevant products.

The system follows a pipeline-based architecture where each stage performs a specific transformation on the data before passing it to the next stage. This structured flow ensures modularity, scalability, and ease of debugging.

Detailed Data Flow Description

The data flow begins when a user interacts with the frontend interface. The input can be in the form of text, voice, or an image.

1. User Input Stage

The user provides input through the frontend interface. This input may be:

1. A text query
2. A voice command
3. An uploaded image

If the input is voice-based, it is first converted into text using speech recognition. If the input is in a regional language, it is translated into English using a frontend translation layer.

2. Preprocessing Stage

Once the input is converted into text (if necessary), it undergoes preprocessing. This includes:

1. Removing unnecessary characters
2. Normalizing text (lowercasing, cleaning)
3. Preparing input for embedding generation

This stage ensures consistency in input format.

3. Embedding Generation Stage

The preprocessed input is converted into embeddings:

1. Text queries → Sentence Transformer embeddings
2. Image inputs → OpenCLIP embeddings

These embeddings represent the semantic or visual meaning of the input.

4. FAISS Retrieval Stage

The generated embeddings are passed to the FAISS index, where similarity search is performed. The index retrieves the top K most similar embeddings from the dataset.

These embeddings correspond to product entries.

5. Hybrid Ranking Stage

The retrieved results are further refined using a hybrid ranking mechanism. This stage combines:

1. Semantic similarity score
2. Keyword matching score
3. Attribute-based filtering

This ensures that the most relevant products are ranked higher.

6. LLM Explanation Stage

The top-ranked products are passed to the LLM module. The model generates explanations describing why the product matches the user query.

This step follows a RAG-inspired approach, ensuring that responses are grounded in actual product data.

7. Output Stage

Finally, the processed results are sent back to the frontend and displayed to the user. The interface presents:

1. Product details
2. Images
3. Rankings

4. Explanations

Data Flow Representation

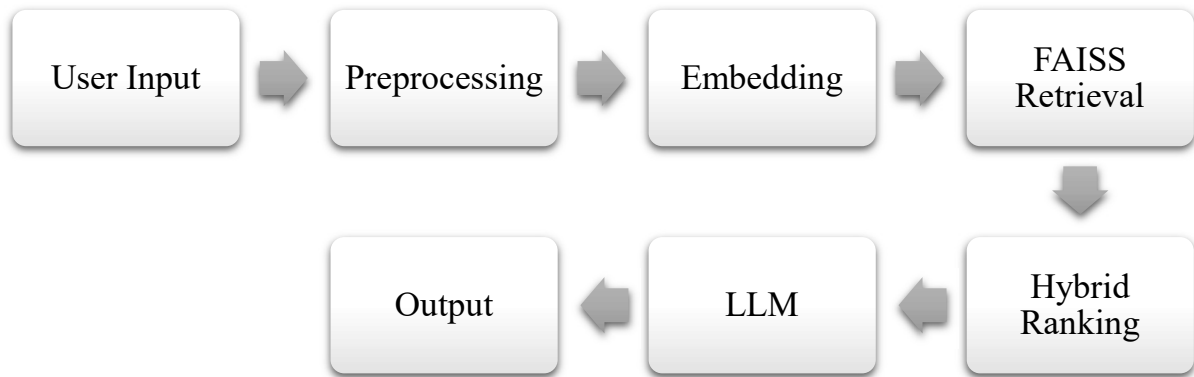


Figure 4.1 Data Flow Representation

Advantages of Data Flow Design

1. Modular processing
2. Efficient handling of multimodal data
3. Easy debugging and scalability
4. Supports real-time response

4.3 Procedural Design

Procedural design describes the step-by-step execution of system operations. It defines how different modules interact to process user queries and generate results.

4.3.1 Text-Based Search Procedure

When a user enters a text query, the system executes a series of steps to retrieve relevant products. Initially, the system preprocesses the query to remove noise and normalize the text. If the query is in a non-English language, it is translated into English.

The processed query is then converted into an embedding using a Sentence Transformer model. This embedding captures the semantic meaning of the query. The embedding is passed to the FAISS index, which retrieves the nearest neighbor embeddings based on similarity.

The retrieved results are then passed to the hybrid ranking module, where they are re-ranked based on multiple criteria.

Finally, the top-ranked products are displayed to the user.

4.3.2 Image-Based Search Procedure

In image-based search, the user uploads an image. The system processes the image using the OpenCLIP model to extract visual features. These features are converted into an embedding, which is then compared with stored embeddings in the FAISS index.

The system retrieves visually similar products and displays them to the user. This procedure enables users to search for products based on appearance rather than textual descriptions.

4.3.3 Hybrid Ranking Procedure

The hybrid ranking procedure is a key component of the system. It improves retrieval accuracy by combining multiple scoring mechanisms.

After FAISS retrieval, the system calculates:

1. Semantic similarity score
2. Keyword matching score
3. Attribute-based adjustments

These scores are combined to compute a final ranking score.

The products are then sorted based on this score, ensuring that the most relevant results are displayed first.

4.3.4 LLM Explanation Procedure

After ranking, the system selects the top products and passes their details to the LLM.

The LLM generates explanations based on the retrieved data. To reduce hallucination, the model is constrained to use only provided product information.

The generated explanation is displayed along with the product results.

4.3.5 End-to-End Workflow

The complete system workflow integrates all modules:

1. User provides input
2. Input is processed and normalized

3. Embeddings are generated
4. FAISS retrieves candidates
5. Hybrid ranking refines results
6. LLM generates explanations
7. Results displayed

4.4 User Interface Design

The User Interface (UI) of the proposed system is designed to provide a seamless, interactive, and user-friendly experience. It acts as the communication layer between the user and the underlying AI-powered backend system. The UI is responsible for capturing user inputs, sending requests to the backend, and displaying processed results in an organized manner.

The interface is developed using HTML, CSS, and JavaScript, ensuring compatibility across different devices and browsers. Special attention is given to responsiveness and usability, allowing users to interact with the system efficiently.

4.4.1 Interface Architecture

The frontend follows a client-server architecture, where the UI communicates with the backend through RESTful APIs.

When a user performs an action such as entering a query or uploading an image, the frontend sends an HTTP request to the backend API. The backend processes the request and returns results in JSON format, which are then dynamically rendered on the UI.

This architecture ensures a clear separation between presentation and processing layers.

4.4.2 Input Handling Mechanisms

The system supports multiple input modes, each handled through specific UI components:

1. Text Input

The primary search bar allows users to enter queries in natural language. The input is captured using JavaScript event listeners and sent to the backend API.

2. Voice Input

The voice search feature is implemented using browser-based speech recognition APIs. When the user clicks the voice button, the system captures audio input, converts it into text, and automatically fills the search field.

3. Image Upload

The UI includes an image upload option that allows users to select an image file. The selected image is sent to the backend using a multipart/form-data request for processing.

4. Multilingual Input

The UI provides a language selection option and supports automatic detection. Queries entered in different languages are translated before being processed.

4.4.3 Output Rendering

The results returned from the backend are displayed in a structured format using a product grid layout.

Each product card includes:

1. Product image
2. Title
3. Price
4. Rating
5. Description
6. Add-to-cart button

The UI dynamically updates the product grid without requiring a full page reload, ensuring a smooth user experience.

4.4.4 State Management

The frontend manages different states such as:

1. Current search results
2. Pagination state
3. Cart items

JavaScript is used to store and update these states dynamically, ensuring that user interactions are reflected immediately on the interface.

4.4.5 Advanced UI Features

The interface includes several advanced features:

1. **Dark Mode:** Implemented using CSS themes to improve visual comfort
2. **Pagination:** Allows navigation through large result sets
3. **Sorting Options:** Enables sorting by relevance or price
4. **Chatbot Interface:** Provides interactive assistance

4.4.6 User Interaction Flow

1. User enters query (text/voice/image)
2. UI sends request to backend
3. Backend processes query
4. Results returned
5. UI renders products
6. User interacts (filter, sort, cart, chatbot)

4.4.7 Importance of UI Design

A well-designed user interface plays a crucial role in the effectiveness of the proposed system. It acts as the bridge between the user and the complex backend processes, ensuring that advanced functionalities are presented in a simple and intuitive manner.

In this system, the UI enables users to interact seamlessly through multiple input modes such as text, voice, and image-based search. Features like multilingual support, product visualization, sorting, pagination, and shopping cart functionality enhance usability and provide a real-world e-commerce experience.

The UI also ensures smooth communication with the backend APIs developed using FastAPI, allowing real-time retrieval and display of results. Dynamic rendering of product data improves responsiveness and reduces user effort.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The implementation of the proposed **Multimodal AI-Powered E-Commerce Product Recommendation System** follows a structured and modular approach, ensuring efficient integration of various components such as data processing, embedding generation, similarity search, and user interaction.

The system is designed using a **pipeline-based architecture**, where each stage performs a specific operation on the input data before passing it to the next stage. This approach improves scalability, maintainability, and debugging capabilities.

5.1.1 Data-Centric Implementation Approach

One of the key approaches adopted in this project is a **data-centric methodology**, where significant effort is invested in preparing and structuring the dataset.

Initially, multiple datasets (approximately 5–7 sources) were collected from different domains such as fashion, electronics, and general e-commerce platforms. These datasets varied in format and structure, requiring extensive preprocessing.

The preprocessing phase involved:

1. Cleaning inconsistent data
2. Handling missing values
3. Standardizing attributes across datasets
4. Merging datasets into a unified structure

A major contribution in this phase was the creation of the **search_text field**, which combines multiple attributes such as title, description, category, and tags. This field plays a crucial role in improving the quality of semantic embeddings.

This data-centric approach ensures that the system performs effectively, as the quality of input data directly impacts retrieval accuracy.

5.1.2 Embedding-Based Retrieval Approach

The system adopts an **embedding-based retrieval approach**, where both textual and visual data are converted into numerical vector representations.

For textual data, the Sentence Transformer model is used to generate semantic embeddings. These embeddings capture the meaning of the text rather than relying on keyword matching.

For image data, the OpenCLIP model is used to generate visual embeddings. These embeddings represent visual characteristics such as color, shape, and texture.

Initially, embedding generation was performed using CPU, which took approximately 5–6 hours. To optimize performance, GPU acceleration was used, significantly reducing processing time to minutes. This optimization was essential for handling large datasets efficiently.

5.1.3 FAISS-Based Retrieval Approach

To enable fast similarity search, the system uses FAISS as the core retrieval engine.

Separate FAISS indexes are created for:

1. Text embeddings
2. Image embeddings

This separation allows independent processing of textual and visual queries.

FAISS enables efficient nearest neighbor search, reducing retrieval time and making the system suitable for real-time applications.

5.1.4 Hybrid Ranking Approach

A hybrid ranking strategy is implemented to improve search accuracy. Instead of relying on a single scoring mechanism, the system combines:

1. Semantic similarity score
2. Keyword matching score
3. Attribute-based filtering

Multiple ranking strategies were experimented with before finalizing this approach. The hybrid method ensures that both contextual meaning and exact keyword matches are considered.

5.1.5 LLM-Based Explanation Approach

The system integrates a Large Language Model (LLM) to generate explanations for recommended products.

Initially, the Mistral model was used, but due to system constraints such as memory usage and performance, it was replaced with the lightweight Phi-3 model.

To reduce hallucination, the system follows a **RAG-inspired approach**, where product data is provided as context to the model. This ensures that generated responses are relevant and grounded in actual data.

5.1.6 API-Based System Integration

The system follows a client-server architecture where the frontend communicates with the backend using RESTful APIs.

FastAPI is used to implement backend endpoints for:

1. Text search
2. Image-based retrieval
3. Chatbot interaction

This approach ensures efficient communication and modular design.

5.1.7 Multimodal Interaction Approach

The system supports multiple input modalities, including:

1. Text input
2. Voice input
3. Image input

This multimodal approach enhances user experience and makes the system more flexible and accessible.

5.2 Coding Details and Code Efficiency

5.2.1 Code Organization

The project is implemented in a modular structure where each functionality is separated into different Python files. For example, `retrieval.py` handles text-based search, `image_retrieval.py` handles image similarity, and `llm.py` is used for generating explanations. This modular approach makes the code easier to maintain and debug. It also allows individual components to be updated without affecting the entire system.

5.2.2 Data Processing Implementation

The dataset is processed using the **Pandas** library, which provides efficient data handling through DataFrames. Multiple datasets collected from different sources are merged into a single unified dataset. During preprocessing, missing values are handled, duplicate entries are removed, and column formats are standardized.

The **re (regular expressions)** library is used to clean textual data by removing unwanted characters and noise. A key feature implemented is the **search_text field**, which combines product title, description, category, and tags. This improves semantic understanding during embedding generation.

5.2.3 Text Embedding Implementation

Text embeddings are generated using the **Sentence Transformers** library, which converts textual data into dense vector representations. The model used processes the **search_text** column and generates embeddings that capture the semantic meaning of the data.

Initially, embeddings were generated using CPU, which took approximately 5–6 hours due to the large dataset size. Later, **PyTorch (torch)** was used to enable GPU acceleration, significantly reducing processing time to a few minutes. Batch processing is also implemented to improve efficiency by processing multiple records at once.

The system uses the Sentence Transformer model (**BAAI/bge-small-en-v1.5**), which is based on the SBERT architecture, to generate dense semantic embeddings for textual data.

Example Code:

```
model = SentenceTransformer('BAAI/bge-small-en-v1.5')  
  
embedding = model.encode(text)
```

5.2.4 Image Embedding Implementation

Image embeddings are generated using the **OpenCLIP** library, which is designed for visual feature extraction. Product images are loaded using the **PIL (Python Imaging Library)** and preprocessed before being passed to the model.

The model converts images into embeddings that represent visual characteristics such as color, shape, and texture. These embeddings allow the system to perform image-based search, where users can upload an image and retrieve visually similar products.

Code Concept:

```
image = Image.open(path)

embedding = model.encode_image(image)
```

5.2.5 FAISS Implementation

The system uses **FAISS (Facebook AI Similarity Search)** to perform efficient similarity search on high-dimensional embeddings. Both text and image embeddings are stored in FAISS indexes.

The embeddings are first converted into **NumPy arrays** for efficient computation and then added to the FAISS index. These indexes are saved using binary files (.bin) and loaded during runtime. FAISS significantly reduces search time by retrieving only the nearest neighbors instead of comparing with the entire dataset.

5.2.6 Retrieval Logic Implementation

The retrieval logic is implemented in the retrieval.py module. When a user enters a query, it is converted into an embedding using Sentence Transformers. This embedding is then passed to the FAISS index to retrieve similar products.

After retrieval, a hybrid ranking mechanism is applied. This combines semantic similarity with keyword matching to improve accuracy. The final results are sorted and returned to the user.

Flow:



Figure 5.1 Retrieval logic

5.2.7 Backend API Implementation

The backend is implemented using **FastAPI**, which provides a high-performance framework for building APIs. It defines endpoints such as `/search`, `/image-search`, and `/chat`.

The frontend communicates with the backend using HTTP requests, and the responses are returned in JSON format. The **CORSMiddleware** is used to allow communication between frontend and backend running on different ports. The **requests** library is also used for API communication and testing.

5.2.8 Multilingual Processing Implementation

Multilingual support is implemented using **langdetect** and **deep_translator (GoogleTranslator)**. The system first detects the language of the user query and then translates it into English before processing.

This allows users to search in different languages while maintaining a unified processing pipeline. The translation layer ensures that embeddings are generated consistently.

1. Detect language:

```
detect(query)
```

2. Translate:

```
GoogleTranslator().translate(query)
```

5.2.9 Data Storage and Serialization

The **pickle** library is used to store embeddings and FAISS indexes. This prevents the need to regenerate embeddings every time the system runs, saving significant computation time.

File handling operations are managed using the **os** module, while **shutil** is used for file operations such as copying and managing image files.

5.2.10 Efficiency Techniques

Several techniques are implemented to improve code efficiency:

- **GPU acceleration (PyTorch)** reduces embedding generation time
- **Batch processing** improves performance for large datasets

- **FAISS indexing** enables fast similarity search
- **Pickle storage** avoids recomputation
- **NumPy arrays** ensure efficient numerical operations

These optimizations ensure that the system performs efficiently even with large-scale data.

5.2.11 Error Handling

Error handling is implemented to ensure system stability and robustness. The system checks for invalid inputs, missing images, and API errors. If an image is not found, it is skipped instead of causing a crash. Similarly, invalid queries are handled by returning appropriate error messages. This ensures that the system continues to function smoothly even in unexpected situations.

Libraries used:

Library	Purpose
pandas	Data preprocessing and dataset handling
numpy	Numerical computations and array operations
sentence-transformers	Generating semantic text embeddings
open_clip	Generating image embeddings for visual search
torch	Deep learning framework and GPU acceleration
faiss	Efficient similarity search and indexing
fastapi	Backend API development
langdetect	Detecting input query language
deep_translator	Translating multilingual queries
PIL (Pillow)	Image loading and preprocessing
pickle	Storing embeddings and model data

requests	API communication and testing
tqdm	Progress tracking during processing

Table 8

Table 5.1: Libraries Used and Their Roles

5.3 Testing Approach

5.3.1 Testing Strategy

The system is tested using a structured approach to ensure correctness and performance. Testing is performed at different stages of development, allowing continuous improvement. The main types of testing include functional testing, accuracy testing, and performance testing. Each type focuses on a specific aspect of the system.

5.3.2 Functional Testing

Functional testing ensures that all features of the system work correctly. Text search is tested using different types of queries, including keywords and full sentences. Image search is tested by uploading product images and verifying whether similar products are retrieved. Voice search is tested to ensure accurate speech-to-text conversion. Multilingual support is tested using queries in different languages. The chatbot is tested to ensure it generates meaningful responses. This ensures that all system functionalities operate as expected.

1. Text Search Testing

Example:

1. Input Query → “red shoes for men”
2. Expected Output → Products related to red-colored men's shoes
3. Observation → System successfully retrieves relevant products based on semantic meaning

2. Image Search Testing

Example:

1. Input → Image of a sneaker
2. Expected Output → Similar sneaker products
3. Observation → System retrieves visually similar products using OpenCLIP embeddings

3. Voice Search Testing

Example:

1. Input → Spoken query “*black jacket*”
2. Process → Converted to text using speech recognition
3. Output → Relevant jacket products displayed

4. Multilingual Testing

Example:

1. Input → “लाल जूते” (Hindi for red shoes)
2. Process → Detected + translated to English
3. Output → Correct product results

5.3.3 Retrieval Accuracy Testing

The accuracy of the system is evaluated using the **Precision@K** metric. This metric measures how many of the top K results are relevant to the query. A custom evaluation function is implemented in `evaluation.py` to calculate this metric. Multiple queries are tested, and the results are analyzed to identify areas of improvement. This helps in refining the ranking algorithm and improving overall accuracy.

1. Example:

1. Query → “*wireless headphones*”
 2. Top 5 results returned
 3. Relevant products = 4
2. Precision@5 calculation:
1. $\text{Precision@5} = 4 / 5 = 0.8$
3. Interpretation:
1. 80% of retrieved results are relevant
 2. Indicates good retrieval performance

This evaluation helps in improving ranking logic and embedding quality

5.3.4 Performance Testing

Performance testing evaluates the efficiency of the system. Key factors such as response time, embedding generation time, and retrieval speed are measured. Initially, embedding generation was slow when using CPU. After switching to GPU, performance improved

significantly. The use of FAISS ensures fast retrieval, making the system suitable for real-time applications. This testing confirms that the system performs efficiently under different conditions.

Performance testing evaluates how efficiently the system performs under different conditions.

1. Embedding Generation

1. CPU Time → ~5–6 hours
2. GPU Time → Few minutes
3. Improvement → Significant performance gain

2. Search Response Time

1. Example:
 1. Query → “smartphone under 20000”
 2. Response Time → 2-3 second
 2. Observation → FAISS enables fast retrieval

3. System Load Handling

1. Multiple queries tested
2. System responds without delay

5.3.5 Edge Case Testing

Edge case testing is performed to ensure that the system can handle unusual inputs. Test cases include empty queries, invalid inputs, missing images, and mixed-language queries. The system is designed to handle these cases gracefully by returning appropriate responses. This improves the robustness and reliability of the system.

1. Empty Query

1. Input → ""
2. Output → Error message or no results
3. System handles gracefully

2. Invalid Image

1. Input → Unsupported file format
2. Output → Error handled without crash

3. Mixed Language Query

1. Input → “red shoes जूते”
2. Output → Correct results after processing

4. Missing Image Path

1. Input → Product without image
2. Output → Skipped or default handling

5.3.6 Iterative Testing and Improvements

Testing is carried out continuously during development. Based on test results, improvements are made to the system. For example, the ranking logic is refined, embedding quality is improved, and performance optimizations are implemented. This iterative approach ensures that the system becomes more accurate and efficient over time.

1. Testing was performed continuously during development
2. Issues identified:
 1. Slow embedding generation
 2. Ranking inaccuracies
3. Improvements made:
 1. Switched to GPU
 2. Improved hybrid ranking logic
 3. Optimized retrieval

Test Scenario	Input	Expected Outcome
Text Search	"black shoes for men"	Relevant products displayed based on semantic meaning
Image Search	Upload sneaker image	Visually similar products retrieved
Voice Search	Spoken "red shirt"	Converted to text and correct results displayed
Multilingual Search	"जूते" (Hindi)	Translated and relevant products shown
Hybrid Retrieval	"analog watch"	Accurate ranking using combined scores

Chatbot Query	"Best shoes for running?"	General shopping guidance provided
Empty Input	""	Error handled gracefully
Invalid Image	Unsupported file	System handles without crash

Table 9

Table 5.2: Testing Scenarios and Expected Outcomes

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Test Reports

The test reports present the results obtained from different functionalities of the system. The system was evaluated using various test cases covering text search, image search, multilingual input, voice input, and chatbot responses. The results demonstrate that the system performs effectively in retrieving relevant products and providing meaningful outputs.

6.1.1 Text Search Results

The text search functionality was tested using different types of queries, including simple keywords and natural language sentences.

1. Example Test Case:

1. Input → “red shoes for men”
2. Process → Query converted into embedding using Sentence Transformers
3. Retrieval → FAISS returns similar products
4. Output → Relevant products displayed

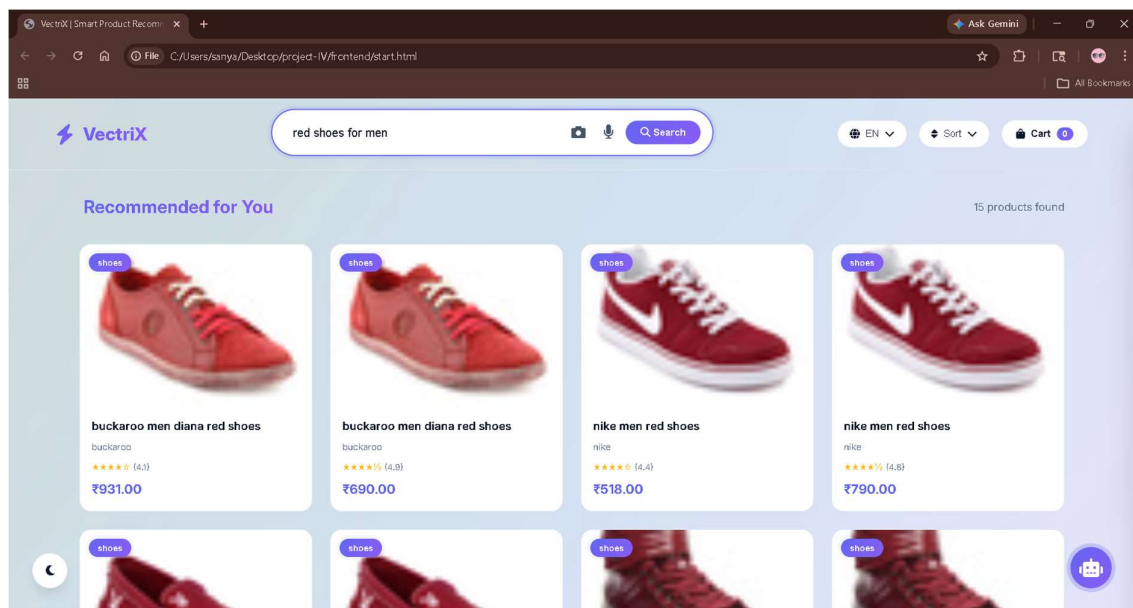


Figure 6.1: Text-based search results for query "red shoes for men"

2. Observation:

1. The system successfully retrieves products based on meaning, not just exact keywords
2. Even slightly different queries return relevant results

6.1.2 Image Search Results

The image search feature allows users to upload an image and retrieve visually similar products.

1. Example Test Case:

1. Input → Image of black shirt
2. Process → Converted into embedding using OpenCLIP
3. Retrieval → FAISS image index used
4. Output → Similar sneakers displayed

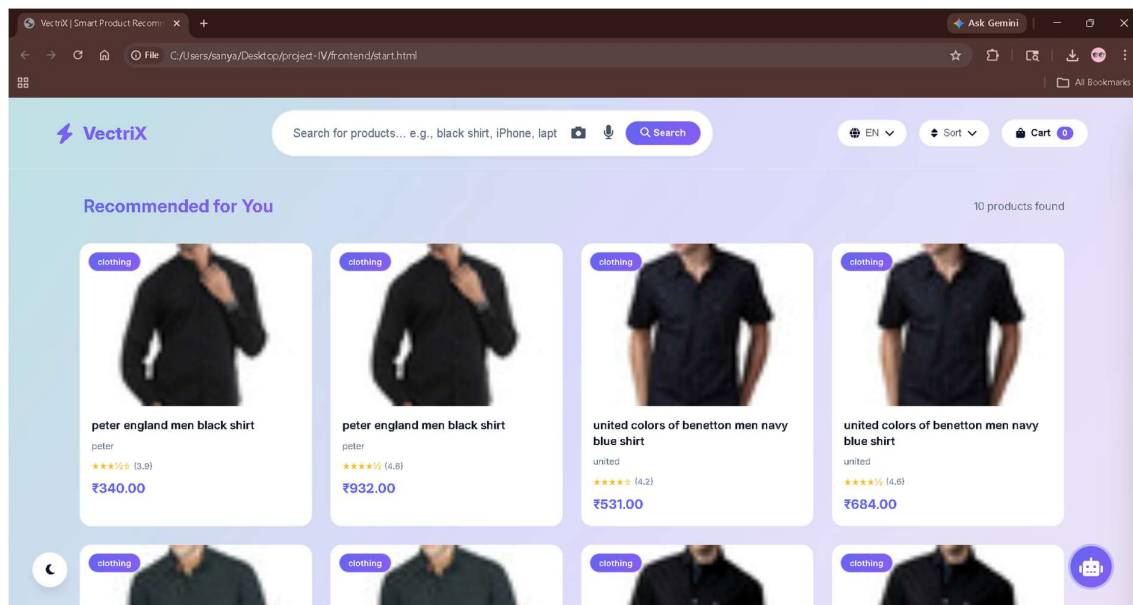


Figure 6.2: Image-based search results using uploaded product image

2. Observation:

1. System successfully matches visual features
2. Useful for users who don't know product names

6.1.3 Multilingual Search Results

The system supports multiple languages through automatic detection and translation.

1. Example Test Case:

1. Input → “झुमके” (Hindi)
2. Process → Detected + translated to English
3. Retrieval → FAISS search
4. Output → Relevant results

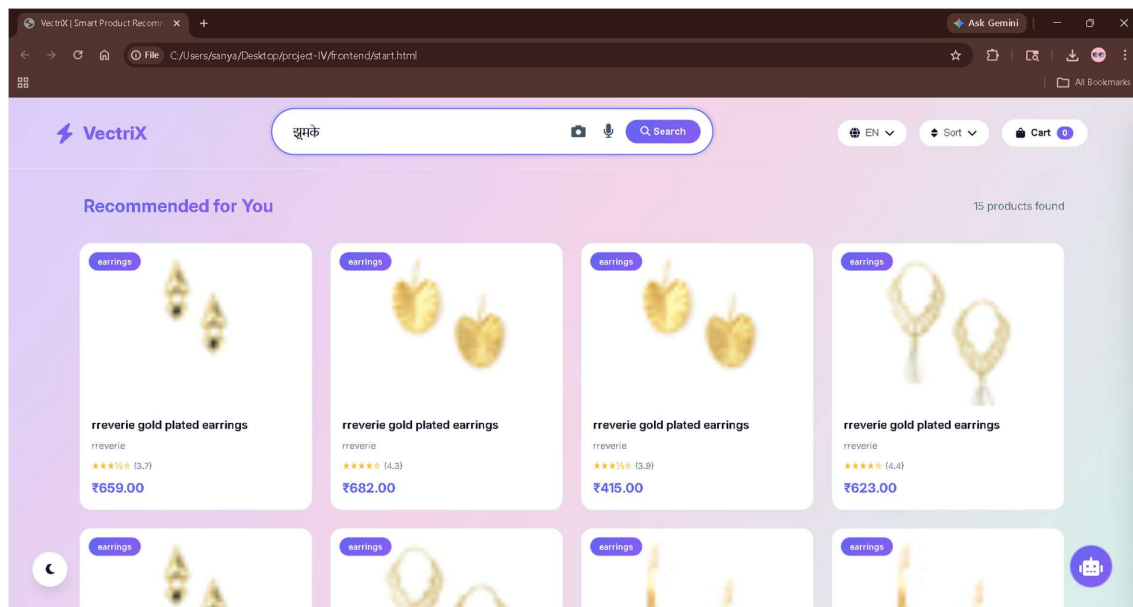


Figure 6.3: Multilingual search results for non-English query

2. Observation:

1. System correctly detects language
2. Translation layer works effectively

6.1.4 Voice Search Results

Voice search improves user interaction by allowing spoken input.

1. Example Test Case:

1. Input → Spoken query “*black jacket*”
2. Process → Converted to text
3. Output → Relevant products

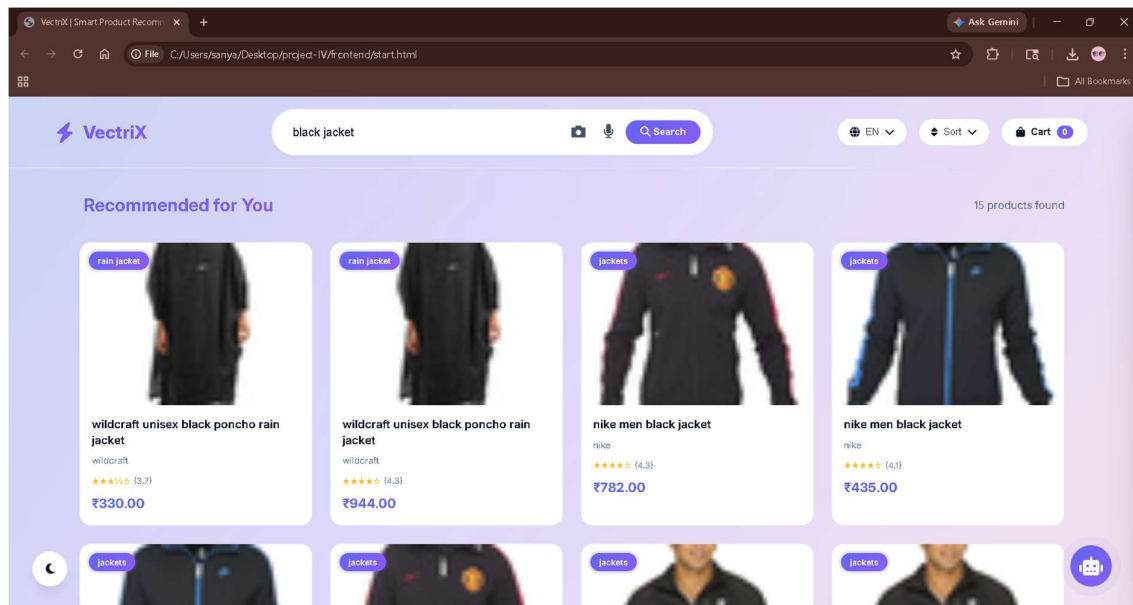


Figure 6.4: Voice-based search functionality

2. Observation:

1. Speech recognition works accurately
2. Improves accessibility

6.1.5 Hybrid Retrieval Results

The system uses hybrid ranking to improve accuracy.

1. Example:
 1. Query → “analog watch”
 2. Semantic similarity + keyword matching applied
 3. Output → Highly relevant products

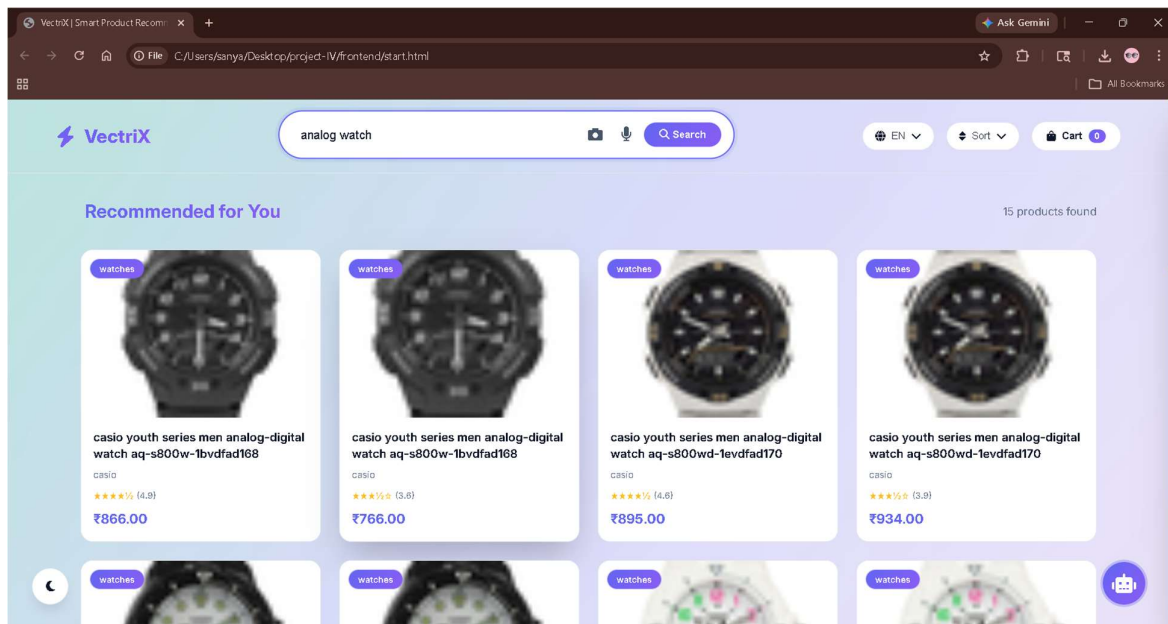


Figure 6.5: Hybrid retrieval showing ranked product results

2. Observation:
 1. Results are more accurate than basic search
 2. Combines meaning + keywords

6.1.6 Best Product Highlighting

The system highlights the most relevant product among the retrieved results. This is determined based on the hybrid ranking score, which combines semantic similarity and keyword matching.

1. Example:

1. Query → “*wireless headphones*”
2. Output → Multiple products shown
3. Top product → Highlighted as best match

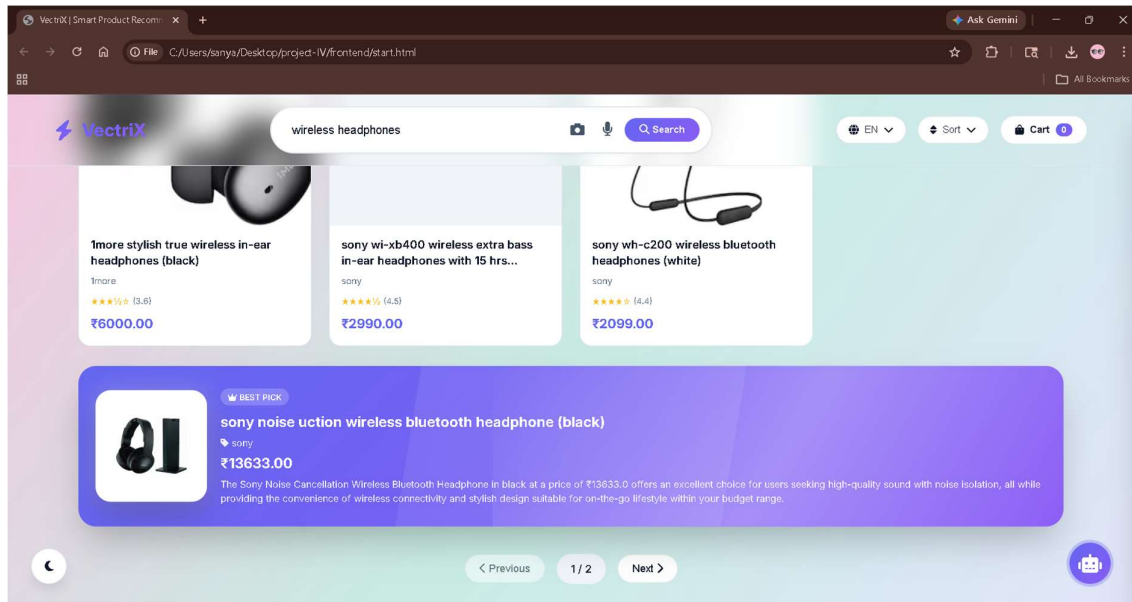


Figure 6.6: Best product highlighted based on ranking

2. Observation:

1. Helps user quickly identify the most relevant item
2. Improves user experience
3. Reduces decision time

6.1.7 Chatbot Results

The chatbot in the system functions as an intelligent shopping assistant and is integrated with the product retrieval module. Unlike a standalone chatbot, it follows a Retrieval-Augmented Generation (RAG) approach, where relevant products are first retrieved based on the user query and then passed to the LLM to generate context-aware responses. This enables the chatbot to provide both general guidance and product-specific recommendations.

1. Example Test Case:

1. User asks → “Which type of shoes are best for running?”
2. Process → Query → FAISS retrieval → relevant products → LLM (Phi-3)
3. Output → Suggestions based on retrieved running shoes

2. Another Example:

1. User asks → “What should I check before buying a laptop?”
2. Output → General buying suggestions with relevant product context

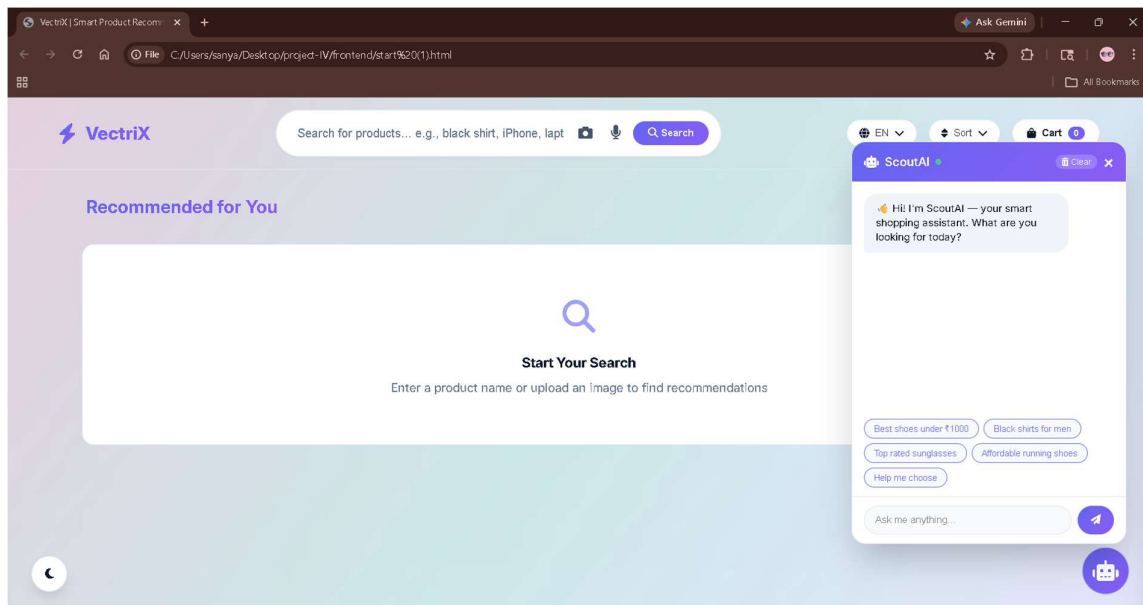


Figure 6.7: Chatbot providing general shopping assistance

3. Observation:

1. Chatbot provides accurate and context-aware responses
2. It retrieves and recommends relevant products based on user queries
3. It works in integration with the search and retrieval system

6.1.8 User Interface Features

1. Product modal

Product Modal View

The product modal displays detailed information about a selected product when the user clicks on it. It provides additional details such as product description, price, rating, and image.

1. Purpose:
 1. Gives detailed view without navigating away
 2. Improves user experience

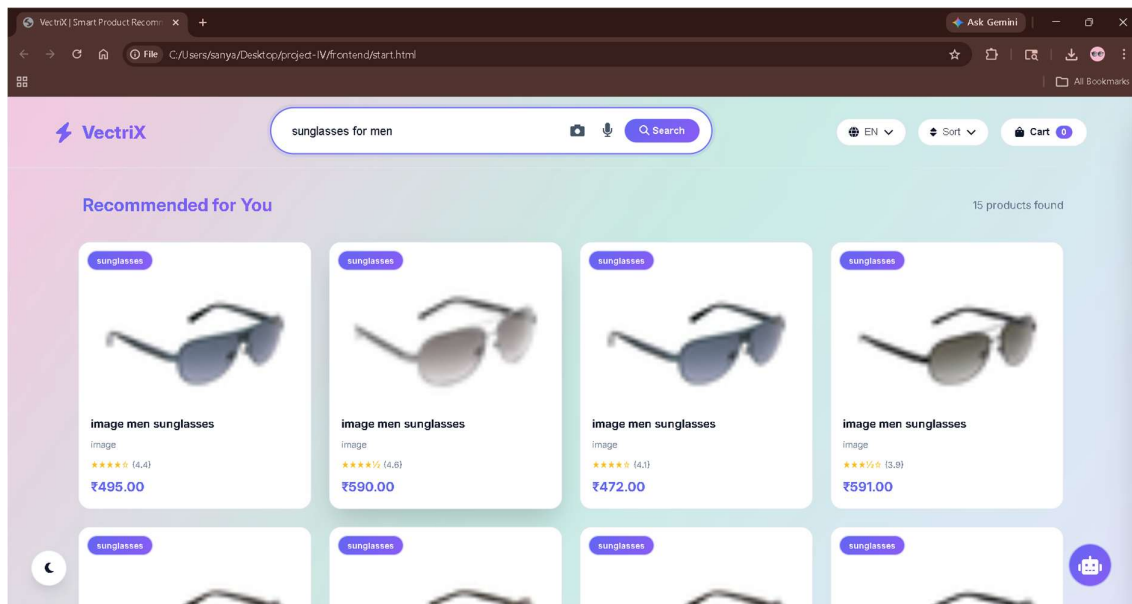


Figure 6.8: Product modal displaying detailed product information

2. Observation:
 1. Users can quickly view product details
 2. Smooth interaction

3. Shopping cart

Shopping Cart Functionality

The system includes a shopping cart feature that allows users to add and manage selected products.

1. Features:

1. Add to cart
2. View cart items
3. Remove items

2. Purpose:

1. Simulates real e-commerce system
2. Enhances usability

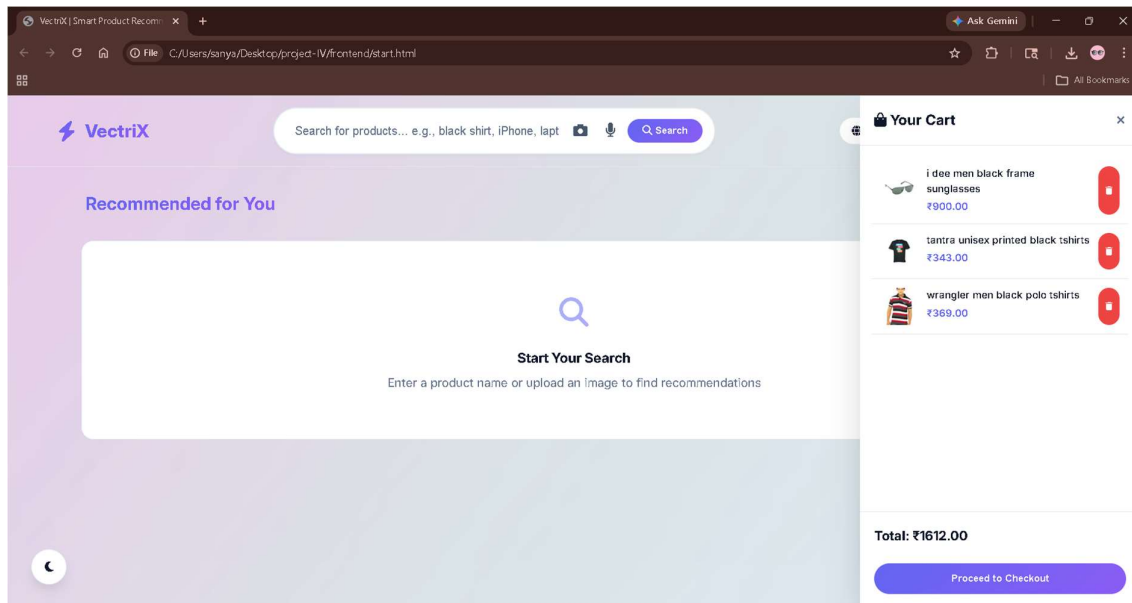


Figure 6.9: Shopping cart displaying selected products

3. Observation:

1. Cart updates dynamically
2. Easy to manage products

3. Dark mode

Dark Mode Interface

The system provides a dark mode option to improve visual comfort and user experience.

1. Features:
 1. Toggle between light and dark theme
 2. Improved readability in low light
2. Purpose:
 1. Enhances UI experience
 2. Modern interface design

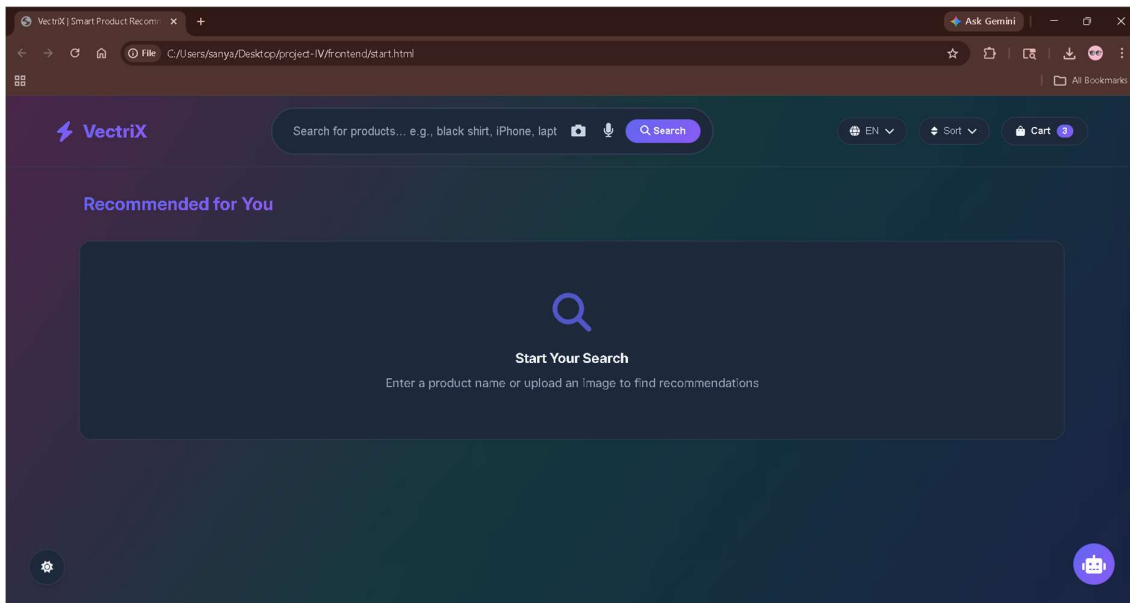


Figure 6.10: Dark mode interface of the application

3. Observation:
 1. UI changes smoothly
 2. Consistent design maintained

The retrieval performance of the system is evaluated using **the Precision@K** metric, as discussed in Section 5.3.3. The evaluation results are presented in **Table 6.1**.

Query	K Value	Relevant Results	Precision@K
"red shoes"	5	4	0.8
"wireless headphones"	5	4	0.8
"black jacket"	5	3	0.6
"smartphone under 20000"	5	4	0.8
"sports shoes"	5	5	1.0

Table 10

Table 6.1: Precision@K Evaluation Results

6.2 Discussion

The results clearly demonstrate that the system is capable of effectively handling multimodal inputs, including text, image, and voice-based queries. The integration of these input modalities provides flexibility and enhances user interaction with the system.

The retrieval performance, as shown in Table 6.1, indicates that the system achieves high accuracy for most queries. The Precision@5 values range between 0.6 and 1.0, with several queries achieving 0.8 and above, which reflects strong retrieval effectiveness. This demonstrates that the combination of semantic embeddings and hybrid ranking significantly improves the relevance of search results compared to traditional keyword-based methods.

The use of FAISS enables fast similarity search, ensuring real-time response even with high-dimensional embeddings. Additionally, the hybrid ranking mechanism enhances result quality by combining semantic similarity with keyword matching, leading to better ranking accuracy.

The multilingual feature increases accessibility by allowing users to interact in different languages, while the chatbot improves user engagement by providing general guidance and explanations.

However, certain limitations are observed. The system performance depends on the quality of embeddings and dataset consistency. Some queries, such as “black jacket,” show relatively lower precision (0.6), indicating scope for improvement in ranking logic. Additionally, image retrieval performance may vary depending on image quality.

Overall, the system demonstrates strong performance in terms of accuracy, efficiency, and usability, making it a reliable solution for modern e-commerce product recommendation systems.

CHAPTER 7

CONCLUSIONS

7. Conclusion

The proposed Multimodal AI-Powered E-Commerce Product Recommendation System successfully demonstrates how modern artificial intelligence techniques can be applied to improve product search and recommendation systems. The system integrates multiple technologies such as Natural Language Processing, Computer Vision, similarity search using FAISS, and Large Language Models to create an intelligent and user-friendly platform.

A major contribution of this project is the implementation of a **multimodal search system**, where users can interact using text, images, and voice. This improves accessibility and provides flexibility compared to traditional systems that rely only on keyword-based search. The use of **Sentence Transformers** enables semantic understanding of queries, allowing the system to retrieve results based on meaning rather than exact matches.

Another key achievement is the use of **OpenCLIP** for image-based search, which allows users to find products based on visual similarity. This is particularly useful in e-commerce scenarios where users may not know the exact name of a product. The integration of **FAISS** ensures that similarity search is performed efficiently, enabling fast response times even with large datasets.

The project also highlights the importance of **data engineering**, as multiple datasets were collected, cleaned, and merged into a unified structure. The creation of engineered features such as `search_text` significantly improved the quality of embeddings and retrieval accuracy. Additionally, performance optimization techniques such as **GPU acceleration** were implemented, reducing embedding generation time from several hours to minutes.

The system provides several practical features, including multilingual search, voice input, image search, shopping cart functionality, dark mode, and a chatbot for shopping assistance. These features make the system more interactive and closer to real-world e-commerce platforms.

Overall, the project successfully achieves its objective of building an intelligent, efficient, and scalable e-commerce search system that enhances user experience and improves retrieval accuracy.

7.1 Limitations of the System

Although the system performs effectively, there are certain limitations that need to be considered.

1. **Dataset Dependency**

The performance of the system depends heavily on the quality and diversity of the dataset. If the dataset contains incomplete or inconsistent information, it may affect retrieval accuracy.

2. **Image Quality Dependency**

The effectiveness of image search depends on the quality of images. Poor-quality or inconsistent images may reduce the accuracy of visual similarity matching.

3. **Limited LLM Capability**

The system uses a lightweight LLM (Phi-3) due to hardware constraints. While it provides reasonable responses, it may not generate highly detailed or context-rich outputs compared to larger models.

4. **Chatbot Limitations**

The chatbot is integrated with the retrieval system; however, its performance depends on the quality of retrieved results and the underlying LLM. In some cases, responses may be limited by incomplete context, retrieval inaccuracies, or model constraints, affecting the depth of recommendations.

5. **Translation Limitations**

Multilingual support relies on translation tools, which may not always provide accurate translations for complex or mixed-language queries.

6. **Scalability Constraints**

The system is tested on a moderate-sized dataset. For very large datasets, additional optimization techniques such as distributed indexing may be required.

7. **Limited Personalization**

The system does not currently use user history or preferences, which limits its ability to provide personalized recommendations.

7.2 Future Scope of the Project

7.2.1 Integration of Advanced Large Language Models (LLMs)

The current system uses a lightweight model (Phi-3) due to hardware limitations, which provides basic conversational capabilities. In the future, more advanced LLMs such as GPT-based or LLaMA-based models can be integrated to enhance system intelligence.

These advanced models are capable of understanding complex user queries, generating more detailed responses, and supporting conversational interactions. This would allow the system to evolve from a simple assistant into a more intelligent shopping guide. Additionally, advanced LLMs can improve explanation quality and provide better context-aware responses.

Benefit: Improves user interaction, response quality, and overall system intelligence.

7.2.2 Implementation of Real-Time Recommendation Systems

Currently, the system retrieves products based on query similarity without considering user behavior. In the future, real-time recommendation systems can be implemented to provide personalized suggestions.

This can be achieved using techniques such as collaborative filtering, deep learning-based recommendation models, and session-based recommendation systems. These methods analyze user behavior, browsing history, and preferences to generate dynamic recommendations. This will allow the system to adapt to individual users and provide more relevant results.

Benefit: Enhances personalization, improves relevance, and increases user engagement.

7.2.3 Use of Vector Databases Instead of FAISS

FAISS is efficient for local and medium-scale applications, but it has limitations in handling large-scale distributed systems. In future implementations, vector databases such as Pinecone, Weaviate, or Milvus can be used.

These databases provide scalable storage, real-time updates, and cloud-based deployment capabilities. They also support distributed architectures, making them suitable for handling millions of embeddings. This would allow the system to scale efficiently for commercial applications.

Benefit: Enables scalability, real-time updates, and large-scale deployment.

7.2.4 Multimodal Deep Learning Models

The current system processes text and image data separately. In the future, advanced multimodal models can be used to jointly understand both text and images.

Models such as advanced CLIP variants or multimodal transformers can better align textual and visual information. This would improve the accuracy of search results, especially when users provide mixed inputs such as text + image queries. It would also enhance the system's ability to understand complex product descriptions.

Benefit: Improves accuracy and enables deeper understanding of multimodal data.

7.2.5 End-to-End Retrieval-Augmented Generation (RAG)

The current system follows a simplified RAG-inspired approach. In the future, a complete end-to-end RAG pipeline can be implemented.

In such a system, retrieval and generation are tightly integrated, allowing the model to generate responses directly based on retrieved data. Tools such as LangChain and LlamaIndex can be used to build this pipeline. This would reduce hallucination and ensure that responses are grounded in actual data.

Benefit: Improves reliability, reduces hallucination, and enhances response quality.

7.2.6 Advanced Multilingual Models

Currently, the system uses translation-based multilingual support. In the future, advanced multilingual models such as mBERT or XLM-R can be used.

These models can directly process queries in multiple languages without requiring translation. This reduces dependency on translation tools and improves accuracy, especially for complex or mixed-language queries. It also enhances the system's ability to handle diverse linguistic inputs.

Benefit: Improves multilingual accuracy and reduces translation errors.

7.2.7 Deployment Using Cloud and Microservices Architecture

The current system is designed for local execution. In the future, it can be deployed using cloud platforms such as AWS, Azure, or Google Cloud.

Using microservices architecture with tools like Docker and Kubernetes, different components of the system (search, embeddings, API, chatbot) can be deployed independently. This improves scalability, reliability, and maintainability. It also allows the system to handle large user traffic efficiently.

Benefit: Enables scalable, reliable, and production-ready deployment.

7.2.8 Integration with Computer Vision Enhancements

The current image search is based on embedding similarity. In the future, advanced computer vision techniques such as object detection and image segmentation can be added.

Technologies like YOLO can be used to detect specific objects within images, while segmentation can identify product regions. This would improve the precision of image search and allow more detailed visual understanding.

Benefit: Enhances visual search accuracy and enables more precise results.

7.2.9 Personalization Using User Data

Currently, the system does not consider user-specific preferences. In the future, personalization can be implemented using user data.

By analyzing user behavior, browsing history, and purchase patterns, the system can provide personalized recommendations. Machine learning models can be used to adjust ranking based on user preferences. This would make the system more adaptive and user-centric.

Benefit: Improves user satisfaction and recommendation relevance.

7.2.10 Mobile and Edge AI Integration

The system can be extended to mobile platforms using technologies such as Flutter or React Native. This would make the application more accessible to users.

Edge AI models can also be used to perform computations directly on devices, reducing dependency on servers. Features like camera-based product search can be integrated into mobile applications, allowing users to search products instantly.

Benefit: Increases accessibility and enables real-time mobile interaction.

7.2.11 Voice AI and Conversational Commerce

Voice interaction can be enhanced using advanced speech and conversational AI technologies.

Instead of simple voice-to-text conversion, the system can support full conversational voice interactions. Users can interact with the system naturally, ask questions, and receive responses through voice. This would enable hands-free shopping and improve accessibility.

Benefit: Provides natural interaction and improves user convenience.

CHAPTER 8
APPENDICES

8.1 APPENDIX 1: DATASET SAMPLE

Title: AI-Powered Multimodal E-Commerce Search System using FAISS and Hybrid Retrieval

8.1.1 Dataset Overview

The dataset used in this project is created by combining multiple publicly available e-commerce datasets from domains such as fashion products, electronics, and general retail items. The data is preprocessed and structured to support multimodal search, including both textual and visual information.

8.1.2 Sample Dataset Records

Table A: Basic Product Information

product_id	title	category	brand	price	gender	color
63699	catwalk women blue flats	heels	catwalk	835	Men	Blue
48528	forever new women black skirts	skirts	forever	858	Men	Black
56280	timberland men casual black wallets	wallets	timberland	727	Men	Black
80442	catwalk women silver heels	heels	catwalk	490	Men	Silver
97415	samsung galaxy s9	Mobile	samsung	37900	unisex	Multi-color

Table 8.1: Basic Product Information

Table A2: Additional Attributes

product_id	rating	price_range	category_clean
63699	4.9	low	shoes

48528	5.0	low	skirts
56280	4.5	low	wallets
80442	3.7	low	shoes
97415	3.8	high	electronics

Table 8.2: Additional Attributes

Table A3: Engineered Features (Used for Retrieval)

product_id	tags	search_text
63699	catwalk heels blue	catwalk women blue flats ...
48528	forever skirts black	forever new women black skirts ...
56280	timberland wallets black	timberland men casual black wallets ...
80442	catwalk heels silver	catwalk women silver heels ...
97415	samsung mobile multi-color	samsung galaxy s9 ...

Table 8.3: Engineered Features

The above tables represent a sample of the dataset used in the system. The dataset contains both original attributes and engineered features. Basic attributes are used for display and filtering, while engineered features such as search_text and tags are used for generating embeddings and improving retrieval accuracy.

8.1.3 Dataset Attributes Description

1. **product_id**: Unique identifier for each product
2. **title**: Product name
3. **description**: Detailed product information
4. **category**: Product category
5. **brand**: Manufacturer or brand name

6. **price:** Cost of the product
7. **color:** Product color
8. **rating:** Average user rating

8.1.4 Preprocessing Steps

The dataset undergoes several preprocessing steps:

1. Removal of missing and inconsistent values
2. Standardization of categories and brand names
3. Creation of search_text by combining title, description, and category
4. Generation of price ranges for filtering
5. Cleaning and normalization of text data

8.2 APPENDIX 2: CODE SNIPPETS

Title: AI-Powered Multimodal E-Commerce Search System using FAISS and Hybrid Retrieval

8.2.1 Text Embedding Generation

```
from sentence_transformers import SentenceTransformer
import pandas as pd

model = SentenceTransformer('BAAI/bge-small-en-v1.5')
df = pd.read_csv('products.csv')

embeddings = model.encode(df['search_text'].tolist())
```

8.2.2 Image Embedding using OpenCLIP

```
import open_clip
import torch
from PIL import Image

model, preprocess = open_clip.create_model_and_transforms('ViT-B-32', pre
trained='openai')

image = preprocess(Image.open("image.jpg")).unsqueeze(0)
with torch.no_grad():
    image_features = model.encode_image(image)
```

8.2.3 FAISS Index Creation

```
import faiss
import numpy as np

dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(embeddings))
```

8.2.4 FastAPI Backend Endpoint

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/search")
def search(query: str):
    results = retrieve_products(query)
    return results
```

8.2.5 Language Translation

```
from deep_translator import GoogleTranslator
from langdetect import detect

lang = detect(query)
translated = GoogleTranslator(source='auto', target='en').translate(query)
```

8.3 APPENDIX 3: API DETAILS

Title: AI-Powered Multimodal E-Commerce Search System using FAISS and Hybrid Retrieval

8.3.1 API Overview

The system uses FastAPI to provide backend services for handling search queries, image-based retrieval, and chatbot interaction.

8.3.2 Text Search API

Endpoint:

1. GET /search?query=<user_input>

Description:

1. Accepts user query
2. Returns relevant products based on semantic similarity

8.3.3 Image Search API

Endpoint:

1. POST /image-search

Description:

1. Accepts uploaded image
2. Returns visually similar products

8.3.4 Chatbot API

Endpoint:

1. GET /chat

Description:

1. Accepts user message
2. Retrieves relevant products using FAISS
3. Passes product context to LLM

4. Returns context-aware, LLM-generated response

8.3.5 Response Format

```
{  
  "product_id": 101,  
  "title": "Black Running Shoes",  
  "price": 1999,  
  "score": 0.89  
}
```

8.3.6 Error Handling

1. Invalid input handled gracefully
2. Missing query returns error message
3. Unsupported image formats rejected

CHAPTER 9

REFERENCES

REFERENCES

1. Reimers, N. and Gurevych, I. (2019) *Sentence-BERT: Sentence Embeddings using Siamese BERT Networks*, Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 3982–3992.
2. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G. and Sutskever, I. (2021) *Learning Transferable Visual Models from Natural Language Supervision*, Proceedings of the International Conference on Machine Learning (ICML).
3. Johnson, J., Douze, M. and Jégou, H. (2017) *FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors*, Facebook AI Research.
4. Rusum, G.P. and Anasuri, S. (2024) *Vector Databases in Modern Applications: Real-Time Search, Recommendations, and Retrieval-Augmented Generation (RAG)*, Journal of Artificial Intelligence and Data Systems.
5. Medisetty, A. (2023) *AI-Curated Embeddings: A Semantic Approach to Structuring and Indexing Vector Databases*, International Journal of Advanced Computing Research.
6. Ali, O. (2025) *Retrieval-Augmented Generation for Intelligent Querying of Databases and Documents*, Journal of Intelligent Information Systems.
7. Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, Proceedings of NAACL-HLT, pp. 4171–4186.
8. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) *Attention Is All You Need*, Advances in Neural Information Processing Systems (NeurIPS).
9. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020) *Transformers: State-of-the-Art Natural Language Processing*, Proceedings of EMNLP.